

Projekt: Krasnal

Eryk Daczyński
Małgorzata Maćkowiak
Jeremiasz Wacławski
Aleks Zieliński

June 1, 2026

Contents

1	Informacje wstępne	4
1.1	Treść projektu	4
2	Wczytywanie danych	5
2.1	Format pliku wejściowego	5
2.2	DataLoader	5
2.2.1	Kod źródłowy	5
3	Funkcje	8
3.1	det()	8
3.2	EPS.h	8
3.2.1	cmpDouble()	8
3.2.2	sgn()	8
3.2.3	Kod źródłowy	8
3.3	is_point_on_segment()	9
3.4	do_segments_intersect()	9
3.5	distance()	10
3.6	is_point_in_polygon()	10
3.7	graham_scan()	11
3.8	KMP()	12
4	Struktury	13
4.1	Point	13
4.2	Huffman.cpp	14
4.2.1	HuffmanNode	15
4.2.2	HuffmanCode	15
4.2.3	buildHuffmanTree()	15
4.2.4	generateHuffmanCodes()	15
4.2.5	Huffman()	16
4.2.6	HuffmanPrint()	16
4.2.7	HuffmanEquals()	16
4.2.8	HuffmanSort()	16
4.2.9	Kod źródłowy	16
4.3	Edge.h	18
4.4	Graph.cpp	18
4.5	Kopalnia.cpp	19
4.6	Krasnal.cpp	21
4.7	MinCostMaxFlow.cpp	23
4.8	SegmentTree.cpp	25
4.8.1	build()	25
4.8.2	query()	25
4.8.3	update()	25
4.9	Straznik.cpp	27
5	Wyszukiwanie wzorca	27
5.1	Funkcje pomocnicze	27
5.2	Jak działa	28
5.3	Kod źródłowy	28

6	Kompresja i dekompresja	30
6.1	Funkcje pomocnicze	30
6.2	Część wspólna	30
6.3	Kompresja	31
6.4	Dekompresja	31
6.5	Kod źródłowy	31
7	Solvery	35
7.1	BorderPatrolSolver	35
7.1.1	calculateConvexHull()	35
7.1.2	calculatePatrolDistance()	35
7.1.3	Kod źródłowy	35
7.2	WorkAssignmentSolver	36
7.2.1	Model kosztu	36
7.2.2	Wynik	36
7.2.3	Kod źródłowy	36
7.3	GuardCommandSolver	41
7.3.1	Rozstrzyganie remisów	41
7.3.2	Aktualizacja	41
7.3.3	Kod źródłowy	41
8	Integracja programu	43
8.1	KingdomManager	43
8.1.1	processWorkAssignment()	44
8.1.2	processBorderPatrol()	44
8.1.3	processGuardCommand()	44
8.1.4	Kod źródłowy	44
9	Testy funkcji	47
9.1	test_kmp.cpp	47
9.2	test_det.cpp	47
9.3	test_is_point_on_segment.cpp	49
9.4	test_do_segments_intersect.cpp	50
9.5	test_is_point_in_polygon.cpp	51
9.6	test_graham_scan.cpp	51
10	Testy struktur	54
10.1	test_data_loader.cpp	54
10.2	test_min_cost_max_flow.cpp	58
10.3	test_point.cpp	59
10.4	test_huffman.cpp	61
10.5	test_segment_tree.cpp	62
10.6	test_class_BorderPatrol.cpp	64
10.7	test_work_assignment_solver.cpp	65
10.8	test_guard_command_solver.cpp	67
10.9	Uruchamianie testów	68
11	Prezentacja działania	69
11.1	Główny przepływ programu	69
11.2	Przykładowy wynik dla Dane/dane.txt	69

1 Informacje wstępne

1.1 Treść projektu

Pracując nad poniższym projektem, należy: odfiltrować informacje istotne od nieistotnych sformalizować problem i podać jego specyfikację; do rozwiązania wykorzystać odpowiednie struktury danych oraz algorytmy; opisać teoretycznie rozwiązanie; przeanalizować jego optymalność oraz złożoność czasową/pamięciową; zaimplementować rozwiązanie w dowolnym języku programowania; przeprowadzić testy na różnych przykładach.

Problem

Odkąd królowa Śnieżka wraz z księciem z bajki rządzą krasnoludkami, ich królestwo nabiera siły i rośnie. Obecnie zamieszkuje je o wiele więcej niż tylko siedmiu krasnoludków. Każdy z nich jest specjalistą w swoim fachu i w kopalniach lub wyrobiskach potrafi wydobywać różne minerały. Królowa zna każdego ze swoich poddanych i wie jakie minerały każdy z nich potrafi wydobywać. To ona decyduje o miejscu pracy każdego z krasnoludków, jest to dla niej niełatwym zadaniem. Chcąc dbać o swoich obywateli, zapewniając im dostatek, królowa zleciła badania geologiczne w swoim kraju. Dzięki temu ma informacje o położeniu poszczególnych złóż w swoim kraju i wie, jaka jest ich wydajność (ilu krasnoludków może pracować w tym miejscu). Okazuje się, że najważniejsza jest miłość do pracy, którą się wykonuje. To znaczy, że jeśli krasnoludek pracuje przy materii zgodnej z jego preferencjami, wydobędzie kruszec o stałej wartości, niezależnie czy kopie złoto, węgiel, rudę miedzi, czy inny surowiec. Królowa Śnieżka poprosiła, abyś na podstawie posiadanych przez nią informacji pomógł jej przydzielić pracę każdemu z krasnoludków.

Po rozmowie z księciem okazało się, że jego ukochana bardzo dużo czasu spędza w kuchni, nadzorując gotowanie owsianki dla wygłodniałych brodaczy. Ilość potrzebnej owsianki zwiększa się, gdy krasnoludki muszą pokonać większą odległość z domu do pracy (można przyjąć, że jest to jedyny czynnik). Książę, chcąc ulżyć swojej wybrance, zmierzył i spisał odległości, pomiędzy każdym z wyrobisk i domkiem każdego z krasnoludków. Prosi, abyś uwzględnił te informacje. Przy podziale pracy chciałby, aby sumaryczna odległość, którą każdego dnia muszą pokonać krasnoludki była możliwie jak najmniejsza, przy czym nie możemy pozwolić, żeby spadła wartość produkowanych dóbr.

Niestety nie jest to jedyny problem. Królowa Śnieżka po incydencie ze złą królową jest nieco drażliwa, gdy pojawia się temat jabłek. Nakazała wyciąć wszystkie jabłonie w królestwie i zabroniła spożywania owoców. Krnąbrne krasnoludki starają się jednak wzbogacić swoją dietę o pyszne renety, antonówki, cortlandy, papierówki i ligole. Szmuglują przez granice królestwa pewne ilości tych owoców. W celu egzekucji prawa, zakazującego spożywania jabłek na terenie królestwa, książę codziennie rano musi okrążyć teren, na którym znajdują się wszystkie użytkowane kopalnie. Chciałby, żeby ta codziennie pokonywana odległość była możliwie mała. Gdy i to nie przynosiło skutku, książę wzdłuż trasy swego przejazdu rozstawił dekametrowców (krasnoludki uzbrojone w łuki ustawione co określoną liczbę metrów). Mają one zapobiec przerzucaniu jabłek przez granicę. Ataki takie wciąż zdarzają się często, atakowany jest dany wielometrowy odcinek granicy, przez który leci niezliczona liczba jabłek w jednym momencie. Dekametrowcy nie mogą opuścić swojego odcinka granicy, a najskuteczniejszą obroną przed lecącymi jabłkami okazuje się salwa.

Najgłośniejszy z krasnoludków atakowanej części granicy powinien wydać sekwencję poleceń:

„strzały na cięciwy
- naciągnąć cięciwy
- strzał!”.

Głos krasnoludka jest jego dumą i sprawą drażliwą, niełatwo więc przychodzi im ustalenie najgłośniejszego. Zaproponuj szybki sposób znajdowania dekametrowca z danego kawałka granicy, który winien wydać rozkazy.

Jak widać, królowa i książę muszą wykonać ogrom pracy, wykorzystując skomplikowane działania. Martwią się, że ta wiedza zaginie. Pomóż im zapisać cenną wiedzę w elektronicznych księgach, oszczędzając elektroniczny papier. Zatroszcz się, żeby potomni mogli szybko

wyszukiwać potrzebne informacje.

Co powinny przygotować zespoły studenckie

W ramach projektu należy przygotować rozwiązanie problemu przedstawionego powyżej.

Rozwiązanie powinno składać się z:

- dokumentacji
- plików programów
- opisu poprawności rozwiązania
- raportów przeprowadzonych testów

Wyniki pracy powinny być zapisywane z wykorzystaniem systemu Git (prowadzący powinien mieć dostęp do repozytorium). Projekt będzie oceniany zgodnie z informacjami przedstawionymi na USOS. Każda osoba będzie musiała wypełnić dwie ankiety: opisującą jej wkład w rozwiązanie problemu oraz oceniającą kompletność rozwiązania problemów przez zespół, do którego należy.

2 Wczytywanie danych

2.1 Format pliku wejściowego

Dane wejściowe są przechowywane w pliku `Dane/dane.txt`. Plik jest podzielony na trzy sekcje:

- **KRASNALE** – identyfikator krasnala, współrzędne domu, preferowany surowiec, liczba umiejętności i lista umiejętności,
- **KOPALNIE** – identyfikator kopalni, współrzędne, wydobywany surowiec i pojemność,
- **STRAZNICY** – identyfikator strażnika i jego głośność.

Wczytane dane są zapisywane w strukturze *InputData*, która przechowuje trzy wektory: krasnali, kopalni i strażników.

2.2 DataLoader

Klasa *DataLoader* odpowiada za odczytanie pliku wejściowego i przekształcenie kolejnych linii na obiekty programu. Podczas czytania zapamiętywana jest aktualna sekcja pliku, dzięki czemu każda linia jest interpretowana jako krasnal, kopalnia albo strażnik. Parser sprawdza również podstawowe błędy danych: brak pliku, niepoprawny format linii, ujemną liczbę umiejętności, ujemną pojemność kopalni oraz ujemną głośność strażnika.

2.2.1 Kod źródłowy

```
1 #ifndef INPUT_DATA_H
2 #define INPUT_DATA_H
3
4 #include "../Struktury/Kopalnia.h"
5 #include "../Struktury/Krasnal.h"
6 #include "../Struktury/Straznik.h"
7
8 #include <vector>
9
10 struct InputData {
11     std::vector<Krasnal> dwarves;
12     std::vector<Kopalnia> mines;
13     std::vector<Straznik> guards;
14 };
15
```

```
16 #endif
```

Plik źródłowy 1: /Parser/InputData.h

```
1 #ifndef DATA_LOADER_H
2 #define DATA_LOADER_H
3
4 #include "InputData.h"
5 #include <string>
6
7 class DataLoader {
8 public:
9     InputData loadFromFile(const std::string& filePath) const;
10 };
11
12 #endif
```

Plik źródłowy 2: /Parser/DataLoader.h

```
1 #include "DataLoader.h"
2
3 #include <algorithm>
4 #include <fstream>
5 #include <iostream>
6 #include <sstream>
7 #include <stdexcept>
8 #include <string>
9 #include <unordered_set>
10 using namespace std;
11
12 enum class Section { None, Dwarves, Mines, Guards };
13
14
15 InputData DataLoader::loadFromFile(const std::string& filePath) const
16 {
17     ifstream file(filePath);
18     if (!file.is_open())
19     {
20         throw runtime_error("Failed to open file");
21     }
22
23     Section currentSection = Section::None;
24     InputData data;
25
26     string line;
27     while (getline(file, line))
28     {
29         if (line.empty()) continue;
30
31         if (line.find("KRASNALE") == 0)
32         {
33             currentSection = Section::Dwarves;
34             continue;
35         }
36         if (line.find("KOPALNIE") == 0)
37         {
38             currentSection = Section::Mines;
39             continue;
40         }
41         if (line.find("STRAZNICY") == 0)
42         {
43             currentSection = Section::Guards;
44             continue;
45         }
46
47         //WCZYTYWANIE DANYCH KRASNALA
48         if (currentSection == Section::Dwarves)
49         {
```

```

50     istream iss(line);
51     int id;
52     double x, y;
53     string preferredResource;
54     int skillCount;
55
56     if (!(iss >> id >> x >> y >> preferredResource >> skillCount))
57     {
58         throw runtime_error("Failed to parse line");
59     }
60
61
62     if (skillCount < 0)
63     {
64         throw runtime_error("Negative skill count");
65     }
66
67     vector<string> skills;
68     for (int i = 0; i < skillCount; i++)
69     {
70         string skill;
71         if (!(iss >> skill))
72         {
73             throw runtime_error("Failed to parse line");
74         }
75
76         skills.push_back(skill);
77     }
78
79     data.dwarves.emplace_back(id, Point(x, y), skills, preferredResource,
false);
80
81     }
82
83     //WCZYTYWANIE DANYCH KOPALNI
84     if (currentSection == Section::Mines)
85     {
86         istream iss(line);
87         int id;
88         double x, y;
89         string resourceType;
90         int capacity;
91
92         if (!(iss >> id >> x >> y >> resourceType >> capacity))
93         {
94             throw runtime_error("Failed to parse line");
95         }
96
97         if (capacity < 0)
98         {
99             throw runtime_error("Negative capacity");
100         }
101
102         data.mines.emplace_back(id, Point(x, y), resourceType, capacity, vector<
int>{});
103
104     }
105     //WCZYTYWANIE DANYCH STRAZNIKOW
106     if (currentSection == Section::Guards)
107     {
108         istream iss(line);
109         int id;
110         int loudness;
111
112         if (!(iss >> id >> loudness))
113         {
114             throw runtime_error("Failed to parse line");
115         }

```

```

116         if (loudness < 0)
117         {
118             throw runtime_error("Negative loudness");
119         }
120     }
121     data.guards.emplace_back(id, loudness);
122 }
123 }
124 }
125 }
126 }
127 return data;
128 }
129 }

```

Plik źródłowy 3: /Parser/DataLoader.cpp

3 Funkcje

3.1 det()

Funkcja pozwalająca ustalić położenie punktu p_3 względem wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) > 0$, to p_3 leży po lewej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) < 0$, to p_3 leży po prawej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) = 0$, to punkty p_1, p_2, p_3 są współliniowe

```

1 #include "det.h"
2
3 double det(Point p1, Point p2, Point p3) {
4     // 3 x 3
5     // D = (x2 - x1)(y3 - y1) - (y2 - y1)(x3 - x1)
6
7     /*
8         | 1 x1 y1 | = p1
9         | 1 x2 y2 | = p2
10        | 1 x3 y3 | = p3
11
12        D > 0 => p3 na lewo od p1p2
13        D < 0 => p3 na prawo od p1p2
14        D = 0 => p3 "na" p1p2 (współliniowy)
15
16        */
17
18     return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);
19 }

```

Plik źródłowy 4: /Struktury/Funkcje/det.cpp

3.2 EPS.h

3.2.1 cmpDouble()

$\text{cmpDouble}(a, b)$ sprawdza która liczba jest większa z uwzględnieniem błędu typu liczbowego double

3.2.2 sgn()

$\text{sgn}(x)$ sprawdza znak liczby x , korzystając z $\text{cmpDouble}(x, 0.0)$ [dokładniej, to sprawdza czy liczba jest większa od 0]

3.2.3 Kod źródłowy

```

1 #ifndef EPS_H
2 #define EPS_H
3

```



```

4 constexpr double EPS = 1e-9;
5
6 inline int cmpDouble(double a, double b) {
7     if (a < b - EPS) return -1;
8     if (a > b + EPS) return 1;
9     return 0;
10 }
11
12 inline int sgn(double x) {
13     return cmpDouble(x, 0.0);
14 }
15
16 #endif

```

Plik źródłowy 5: /Struktury/Funkcje/EPS.h

3.3 is_point_on_segment()

W tej funkcji sprawdzamy przynależność punktu P do odcinka p_1p_2

Pierw sprawdzamy czy punkt P jest współliniowy z $\overrightarrow{p_1p_2}$, jeżeli nie to nie należy do odcinka p_1p_2

Następnie jeżeli jest współliniowy, to sprawdzamy kolejno współrzędne x i y czy spełniają warunki odpowiednio:

$$\min(p_1.x, p_2.x) \leq P.x \leq \max(p_1.x, p_2.x)$$

$$\min(p_1.y, p_2.y) \leq P.y \leq \max(p_1.y, p_2.y)$$

Jeżeli spełnione są oba te warunki to punkt P należy do odcinka p_1p_2

```

1 #include "is_point_on_segment.h"
2
3 // Sprawdzenie czy punkt P należy do odcinka p1p2
4 bool is_point_on_segment(Point p1, Point p2, Point P)
5 {
6     // P należy gdy p1 p2 i P są współliniowe -> D = 0
7     if (sgn(det(p1, p2, P)) != 0){
8         return false;
9     }
10    // P należy gdy min(x1,x2) <= x3 <= max(x1,x2)
11    // oraz max(y1, y2) <= y3 <= max(y1, y2)
12    bool x_ok = cmpDouble(std::min(p1.x, p2.x), P.x) <= 0 &&
13                cmpDouble(P.x, std::max(p1.x, p2.x)) <= 0;
14    bool y_ok = cmpDouble(std::min(p1.y, p2.y), P.y) <= 0 &&
15                cmpDouble(P.y, std::max(p1.y, p2.y)) <= 0;
16
17    return x_ok && y_ok;
18 }

```

Plik źródłowy 6: /Struktury/Funkcje/is_point_on_segment.cpp

3.4 do_segments_intersect()

Funkcja sprawdza czy odcinki p_1p_2 i p_3p_4 przecinają się

Obliczamy $D_1 = \det(p_1, p_2, p_3)$, $D_2 = \det(p_1, p_2, p_4)$, $D_3 = \det(p_3, p_4, p_1)$ i $D_4 = \det(p_3, p_4, p_2)$

Jeżeli $D_1 \cdot D_2 < 0$ i $D_3 \cdot D_4 < 0$ to p_1p_2 i p_3p_4 przecinają się

Jeżeli $D_1 = 0$ lub $D_2 = 0$ lub $D_3 = 0$ lub $D_4 = 0$ to sprawdzamy przynależności punktu do prostej (ponieważ $\overrightarrow{p_1p_2}$ i $\overrightarrow{p_3p_4}$ są współliniowe)

```

1 #include "do_segments_intersect.h"
2
3 // Sprawdzenie czy odcinki p1p2 p3p4 się przecinają
4 bool do_segments_intersect(Point p1, Point p2, Point p3, Point p4)
5 {
6     double d1 = det(p1, p2, p3);
7     double d2 = det(p1, p2, p4);
8     double d3 = det(p3, p4, p1);
9     double d4 = det(p3, p4, p2);
10

```

```

11 // Przecinaja sie gdy:
12 // WARUNEK 2: kazdy odcinek przecina prosta wyznaczona przez drugi odcinek
13 if((sgn(d1) * sgn(d2) < 0) && (sgn(d3) * sgn(d4) < 0)) return true;
14
15 // WARUNEK 1: ktorys z koncow odc. nalezy do drugiego odc.
16 // Czy P1 lezy na P3P4 || Czy P2 lezy na P3P4 || Czy P3 lezy na P1P2 || Czy P4
   lezy na P1P2
17 if(is_point_on_segment(p3, p4, p1) || is_point_on_segment(p3, p4, p2) ||
18     is_point_on_segment(p1, p2, p3) || is_point_on_segment(p1, p2, p4)) {
19     return true;
20 }
21 return false;
22 }

```

Plik źródłowy 7: /Struktury/Funkcje/do_segments_intersect.cpp

3.5 distance()

Funkcja oblicza kwadrat odległości punktu a od punktu b według metryki euklidesowej

```

1 #include "distance.h"
2
3 double distance(Point a, Point b) {
4     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
5 }

```

Plik źródłowy 8: /Struktury/Funkcje/distance.cpp

3.6 is_point_in_polygon()

Funkcja implementuje algorytm z wykładu sprawdzający czy punkt p należy do listy wierzchołków (punktów *Point*) wielokąta *polygon*

Ustalmy $u = \min(x : x \in \text{polygon})$, $v = \max(x : x \in \text{polygon})$

Jeżeli $p.x < u$ lub $p.x > v$ to p nie leży w wielokącie *polygon*

Jeżeli p leży na dowolnej krawędzi wielokąta *polygon* to p leży w wielokącie *polygon*

Niech $q = (v + 1, p.y)$ i $l = 0$, i dla każdej krawędzi $p.y$ należy do domknięto-otwartego przedziału współrzędnych y tej krawędzi oraz pq ją przecina, to zwiększ l o

Jeżeli l jest nieparzyste, to punkt p leży w wielokącie *polygon*

```

1 #include "is_point_in_polygon.h"
2
3 bool is_point_in_polygon(Point p, const std::vector<Point>& polygon){
4     double u = polygon[0].x;
5     double v = polygon[0].x;
6     int n = polygon.size();
7     for(int i = 0; i < n; i++){
8         u = std::min(u, polygon[i].x);
9         v = std::max(v, polygon[i].x);
10    }
11
12    if((p.x < u) || (p.x > v)) return false;
13
14    int l = 0;
15
16    Point q = Point(v + 1, p.y);
17
18    for(int i = 0; i < n-1; i++){
19        if(is_point_on_segment(polygon[i], polygon[i+1], p)) return true;
20        if((p.y >= std::min(polygon[i].y, polygon[i+1].y)) && (p.y < std::max(polygon
[i].y, polygon[i+1].y))){
21            if(do_segments_intersect(polygon[i], polygon[i+1], p, q)) l++;
22        }
23    }
24

```

```

25     if(1 % 2 == 1) return true;
26     return false;
27 }

```

Plik źródłowy 9: /Struktury/Funkcje/is_point_in_polygon.cpp

3.7 graham_scan()

Funkcja implementuje algorytm Grahama do wyznaczenia otoczki wypukłej
Pierw sortujemy rosnąco punkty względem $p.y$, gdy $p.y$ taki sam to względem $p.x$
Następnie usuwamy duplikaty (te same punkty) ze zbioru
Jeżeli zbiór zawiera mniej niż 4 unikalne elementy to zwracamy ten zbiór
Następnie sortujemy zbiór rosnąco względem współrzędnych kątowych, kolejno usuwamy bliższe punkty z tą samą współrzędną kątową co dalszy punkt
Tworzymy stos i dodajemy do niego pierwsze trzy punkty oraz dla każdego kolejnego punktu p_i z wcześniej przygotowanej tablicy dopóki następnik i następnik następnika punktu p_i nie jest skretem w lewo to usuwamy element ze stosu, następnie dodajemy p_i
Ostatecznym wynikiem jest zbiór punktów otoczki wypukłej zbioru wejściowego

```

1 #include "graham_scan.h"
2
3 // https://en.wikipedia.org/wiki/Graham_scan
4 std::vector<Point> graham_scan(std::vector<Point> points){
5     /// KROK 1: USUNIECIE DUPLIKATOW I USTAWIENIE NAJBLIŻSZEGO
6     /// PUNKTU JAKO PIERWSZY (NAJPIERW NA Y, W RAZIE REMISU NA X)
7
8     std::sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
9         int cmpY = cmpDouble(a.y, b.y);
10        if (cmpY != 0) return cmpY < 0;
11        return cmpDouble(a.x, b.x) < 0;
12    });
13
14    points.erase(std::unique(points.begin(), points.end(), [](const Point& a, const
15    Point& b) {
16        return cmpDouble(a.x, b.x) == 0 && cmpDouble(a.y, b.y) == 0;
17    }), points.end());
18
19    /// PRZYPADKI, GDY JEST PONIZEJ 3 PUNKTOW
20    if (points.size() < 3) {
21        if (points.size() == 0) return {};
22        if (points.size() == 1 || points.size() == 2) return points;
23    }
24
25    /// KROK 2: SORTUJEMY I DOSTOSOWUJEMY LISTE PUNKTOW ZE WZGLEDU NA WSPOLRZEDNE
26    KATOWE
27
28    /// NAJPIERW SORTUJEMY LISTE PUNKTOW
29    std::sort(points.begin() + 1, points.end(), [&](const Point& pi, const Point& pj)
30    {
31        double d = det(points[0], pi, pj);
32        if (sgn(d) == 0) {
33            int cmpDist = cmpDouble(distance(points[0], pi), distance(points[0], pj))
34            ;
35            if (cmpDist != 0) return cmpDist < 0;
36            int cmpX = cmpDouble(pi.x, pj.x);
37            if (cmpX != 0) return cmpX < 0;
38            return cmpDouble(pi.y, pj.y) < 0;
39        }
40        return sgn(d) > 0;
41    });
42
43    /// USUWAMY Z LISTY BLIZSZE PUNKTY Z TA SAMA WSPOLRZEDNA KATOWA CO DALSZY PUNKT
44    std::vector<Point> furtherPoints;
45    furtherPoints.push_back(points[0]);

```

```

43     for(unsigned int i = 1; i < points.size(); i++){
44         while(i + 1 < points.size() && sgn(det(points[0], points[i], points[i+1])) ==
45             0)
46             i++;
47         furtherPoints.push_back(points[i]);
48     }
49     points = furtherPoints;
50
51     /// KROK 3: Utworzenie stosu i dodanie 3 pierwszych punktów
52
53     if (points.size() < 3) return points;
54
55     std::vector<Point> stack;
56     stack.push_back(points[0]);
57     stack.push_back(points[1]);
58     stack.push_back(points[2]);
59
60     /// KROK 4: Ostateczne dobranie punktów otoczki
61
62     for (unsigned int i = 3; i < points.size(); i++) {
63         while (stack.size() >= 2 && sgn(det(stack[stack.size() - 2], stack.back(),
64             points[i])) <= 0) {
65             stack.pop_back();
66         }
67         stack.push_back(points[i]);
68     }
69     return stack;
70 }

```

Plik źródłowy 10: /Struktury/Funkcje/graham_scan.cpp

3.8 KMP()

Pierw wykonujemy funkcję buildPi na naszym wzorcu, funkcja tworzy tablicę pi (prefiksów wzorca). Ta tablica przechowuje długość nadłuższego prefiksu będącego jednocześnie sufiksem. Funkcja KMP pierw wykonuje buildPi na wzorcu, następnie wykonuje algorytm przy tablicy pi. Algorytm porównuje kolejne znaki wzorca z kolejnymi znakami tekstu i śledzi długość aktualnego dopasowania.

- Jeśli znaki są zgodne, dopasowanie jest rozszerzane.
- Jeśli wystąpi niezgodność, algorytm wykorzystuje wcześniej obliczoną tablicę prefiksów, aby określić najdłuższy fragment wzorca, który nadal może pasować do już przeanalizowanej części tekstu.

Gdy zostanie dopasowany cały wzorec, algorytm zapisuje pozycję jego wystąpienia i kontynuuje wyszukiwanie kolejnych wystąpień, również takich, które częściowo się nakładają.

```

1  #include "KMP.h"
2
3  std::vector<int> buildPi(std::string P){
4      std::vector<int> pi;
5      //unsigned int m = P.size();
6      for(unsigned int i{}; i<P.size(); i++){
7          pi.push_back(0);
8      }
9
10     unsigned int k = 0;
11     for(unsigned int q = 1; q<P.size(); q++){
12         while((k > 0) && P[k] != P[q]){
13             k = pi[k-1];
14         }
15         if(P[k] == P[q]){
16             k++;

```

```

17     }
18     pi[q] = k;
19 }
20
21     return pi;
22 }
23
24 std::vector<int> KMP(std::string T, std::string P){
25     std::vector<int> pi = buildPi(P);
26     unsigned int q = 0;
27     unsigned int n = T.size();
28     unsigned int m = P.size();
29     std::vector<int> shifts = {};
30
31     for(unsigned int i{}; i<n; i++){
32         while((q > 0) && (P[q] != T[i])){
33             q = pi[q-1];
34         }
35         if(P[q] == T[i]){
36             q++;
37         }
38         if(q == m){
39             shifts.push_back(i - m + 1);
40             q = pi[q-1];
41         }
42     }
43     return shifts;
44 }

```

Plik źródłowy 11: /Struktury/Funkcje/KMP.cpp

4 Struktury

4.1 Point

Struktura naszego punktu w układzie kartezjańskim, zawiera ona definicję oraz operatory do porównywania dwóch punktów oraz wykonywania działań na nich

Operator mniejszości: " $p_1 < p_2$ " sprawdza czy punkt p_1 na lewo od p_2 . Jeżeli mają tę samą współrzędną $p_1.x$, to sprawdza czy p_1 jest poniżej p_2

Operator większości: $p_1 > p_2$ sprawdza $p_2 < p_1$

Operator równości: $p_1 == p_2$ sprawdza czy współrzędne x i y są takie same

Operator nierówności: $p_1 != p_2$ [$\neq$] zwraca przeciwieństwo $p_1 == p_2$

Operator mniejszy-równy: $p_1 <= p_2$ [$\leq$] zwraca $p_1 < p_2 \vee p_1 == p_2$

Operator większy-równy: $p_1 >= p_2$ [$\geq$] zwraca $p_1 > p_2 \vee p_1 == p_2$

Operator przypisania: $p_1 = p_2$ przypisuje punktowi p_1 odpowiednio $p_1.x = p_2.x$ oraz $p_1.y = p_2.y$

Wypisanie punktu: $<< p$ wypisuje punkt p w postaci $(p_1.x, p_1.y)$

```

1 #include "Point.h"
2
3 Point::Point(double x, double y) : x(x), y(y) {}
4
5 // Operator mniejszosci
6 bool Point::operator<(const Point& other) const {
7     int cmpX = cmpDouble(x, other.x);
8     if (cmpX != 0) {
9         return cmpX < 0;
10    }
11    return cmpDouble(y, other.y) < 0;
12 }
13
14 // Operator wielkosci
15 bool Point::operator>(const Point& other) const {
16     return other < *this;
17 }

```

```

18
19 // Operator rownosci
20 bool Point::operator==(const Point& other) const {
21     return cmpDouble(x, other.x) == 0 && cmpDouble(y, other.y) == 0;
22 }
23
24 // Operator nierownosci
25 bool Point::operator!=(const Point& other) const {
26     return !(*this == other);
27 }
28
29 // Operator mniejszosci lub rownosci
30 bool Point::operator<=(const Point& other) const {
31     return (*this < other) || (*this == other);
32 }
33
34 // Operator wiekszosci lub rownosci
35 bool Point::operator>=(const Point& other) const {
36     return (*this > other) || (*this == other);
37 }
38
39 // Operator przypisania
40 Point& Point::operator=(const Point& other) {
41     if(this != &other){
42         x = other.x;
43         y = other.y;
44     }
45
46     return *this;
47 }
48
49 // Wypisanie punktu
50 std::ostream& operator<<(std::ostream& stream, const Point& p){
51     stream << "(" << p.x << ", " << p.y << ")";
52     return stream;
53 }

```

Plik źródłowy 12: /Struktury/Point.cpp

4.2 Huffman.cpp

Bardziej niżeli struktura to zbiór oddzielnych funkcji przy użyciu których korzystamy z algorytmu Huffmana do kodowania tekstu (Algorytm Huffmana)

Poniżej zawartość pliku nagłówkowego dla łatwiejszego zrozumienia kodu źródłowego

```

1 #ifndef HUFFMAN_H
2 #define HUFFMAN_H
3
4 #include <memory>
5 #include <queue>
6 #include <vector>
7 #include <string>
8 #include <iostream>
9 #include <algorithm>
10
11 struct HuffmanNode{
12     char c;
13     int freq;
14     bool isLeaf;
15
16     std::unique_ptr<HuffmanNode> left;
17     std::unique_ptr<HuffmanNode> right;
18
19     HuffmanNode() : c(0), freq(0), isLeaf(false), left(nullptr), right(nullptr) {}
20     HuffmanNode(char character, int frequency) : c(character), freq(frequency),
21         isLeaf(true), left(nullptr), right(nullptr) {}
22 };

```

```

22
23 struct HuffmanCode{
24     char c;
25     std::string code;
26
27     HuffmanCode(char character, const std::string& huffmanCode) : c(character), code(
        huffmanCode) {}
28 };
29
30 std::unique_ptr<HuffmanNode> buildHuffmanTree(const std::vector<char>& C, const std::
    vector<int>& F);
31 std::vector<HuffmanCode> generateHuffmanCodes(const std::unique_ptr<HuffmanNode>&
    node, const std::string& prefix = "");
32 std::vector<HuffmanCode> Huffman(const std::vector<char>& C, const std::vector<int>&
    F);
33 void HuffmanPrint(const std::vector<HuffmanCode>& codes);
34 bool HuffmanEquals(const std::vector<HuffmanCode>& a, const std::vector<HuffmanCode>&
    b);
35 std::vector<HuffmanCode> HuffmanSort(const std::vector<HuffmanCode>& codes);
36
37 #endif

```

Plik źródłowy 13: /Struktury/Huffman.h

4.2.1 HuffmanNode

Struktura która przechowuje znak *c*, liczbę wystąpień *freq*, flagę czy jest liściem *isLeaf*, wskaźnik do lewego dziecka *left* oraz do prawego *right*

Dodatkowo konstruktory do sprawnego tworzenia obiektów

4.2.2 HuffmanCode

Struktura która zawiera znak *c* oraz kod jego odpowiedni kod Huffmana *code*

Dodatkowo konstruktory do sprawnego tworzenia obiektów

4.2.3 buildHuffmanTree()

Funkcja, która jak nazwa wskazuje, tworzy drzewo kodowań Huffmana

Definiujemy funkcję porównywania struktur HuffmanNode która sprawdza która jest większa ” > ” według pola *HuffmanNode.freq*

Następnie sprawdzamy czy któraś z tablic wejściowych (tablica znaków lub tablica liczby wystąpień) jest pusta, jak tak to zwracamy pusty wskaźnik

Tworzymy kolejkę priorytetową typu min z priorytetem *HuffmanNode.freq*

Tworzymy struktury *HuffmanNode* na podstawie danych wejściowych i dodajemy ich wskaźniki do kolejki priorytetowej

Dopóki rozmiar kolejki priorytetowej jest większy od 1 to wybieramy dwie struktury o najniższej wartości *freq* (najwyższy priorytet) jako odpowiednio *left* i *right* oraz tworzymy nowy węzeł z wskaźnikami do *left* i *right* i odkładamy go do kolejki

Po wyjściu z pętli jeżeli kolejka jest pusta to zwracamy pusty wskaźnik, a w przeciwnym wypadku zwracamy jedyny element kolejki który jest korzeniem drzewa Huffmana

4.2.4 generateHuffmanCodes()

Funkcja która rekurencyjnie na podstawie struktury *HuffmanNode* i podanego prefixu tworzy kodowanie Huffmana

Tworzymy tablicę *HuffmanCode* i następnie jeżeli dany element jest liściem to dodajemy znak *c* i jego kod *prefix* do tablicy kodów, jeżeli nie jest liściem, to dla lewego dziecka wykonujemy *generateHuffmanCodes* na tym dziecku z tym samym prefixem plus 0 i zwróconą wartość dodajemy do tablicy kodów, a dla prawego dziecka tak samo tylko wywołujemy funkcję

generateHuffmanCodes od prawego dziecka i z prefixem plus 1 oraz dodajemy do tablicy kodów
Ostatecznie zwracamy tablicę kodów

4.2.5 Huffman()

Wywołuje funkcję *buildHuffmanTree* i na jej wyniku wywołuje funkcję *generateHuffmanCodes* i zwraca jej wynik

4.2.6 HuffmanPrint()

Wypisuje *HuffmanCode* w postaci *znak : kod*

4.2.7 HuffmanEquals()

Funkcja sprawdza czy dwa kody Huffmana są równe w sensie mają ten sam znak *c* i ten sam kod Huffmana

4.2.8 HuffmanSort()

Funkcja sortuje tablicę kodów Huffmana alfabetycznie według znaku *c*

4.2.9 Kod źródłowy

```
1 #include "Huffman.h"
2
3 std::unique_ptr<HuffmanNode> buildHuffmanTree(const std::vector<char>& C, const std::
vector<int>& F){
4     auto cmp = [](const std::unique_ptr<HuffmanNode>& a, const std::unique_ptr<
HuffmanNode>& b){
5         return a->freq > b->freq;
6     };
7
8     if((C.size() != F.size()) || C.empty() || F.empty()){
9         return nullptr;
10    }
11
12    std::priority_queue<std::unique_ptr<HuffmanNode>, std::vector<std::unique_ptr<
HuffmanNode>>, decltype(cmp)> pq(cmp);
13
14    for(size_t i = 0; i < C.size(); i++){
15        pq.push(std::move(std::make_unique<HuffmanNode>(C[i], F[i])));
16    }
17
18    while(pq.size() > 1){
19        auto left = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
20        pq.pop();
21        auto right = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
22        pq.pop();
23
24        auto parent = std::make_unique<HuffmanNode>();
25        parent->freq = left->freq + right->freq;
26        parent->isLeaf = false;
27        parent->left = std::move(left);
28        parent->right = std::move(right);
29
30        pq.push(std::move(parent));
31    }
32
33    if(pq.empty()) return nullptr;
34    auto root = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
35    pq.pop();
36
37    return root;
38 }
```



```

39
40 std::vector<HuffmanCode> generateHuffmanCodes(const std::unique_ptr<HuffmanNode>&
41 node, const std::string& prefix){
42     std::vector<HuffmanCode> codes;
43
44     if(node->isLeaf){
45         codes.emplace_back(node->c, prefix);
46     } else{
47         if(node->left) {
48             auto leftCodes = generateHuffmanCodes(node->left, prefix + "0");
49             codes.insert(codes.end(), leftCodes.begin(), leftCodes.end());
50         }
51         if(node->right) {
52             auto rightCodes = generateHuffmanCodes(node->right, prefix + "1");
53             codes.insert(codes.end(), rightCodes.begin(), rightCodes.end());
54         }
55     }
56     return codes;
57 }
58
59 std::vector<HuffmanCode> Huffman(const std::vector<char>& C, const std::vector<int>&
60 F){
61     auto root = buildHuffmanTree(C, F);
62
63     if(root == nullptr){
64         return {};
65     }
66
67     std::vector<HuffmanCode> huffmanCodes;
68     huffmanCodes = generateHuffmanCodes(root);
69
70     return huffmanCodes;
71 }
72
73 void HuffmanPrint(const std::vector<HuffmanCode>& codes){
74     for(const auto& code : codes){
75         std::cout<<code.c<<" ";
76     }
77 }
78
79 bool HuffmanEquals(const std::vector<HuffmanCode>& a, const std::vector<HuffmanCode>&
80 b){
81     if(a.size() != b.size()) return false;
82
83     for(size_t i = 0; i < a.size(); i++){
84         if((a[i].c != b[i].c) || (a[i].code != b[i].code)){
85             return false;
86         }
87     }
88
89     return true;
90 }
91
92 std::vector<HuffmanCode> HuffmanSort(const std::vector<HuffmanCode>& codes){
93     if(codes.empty()) return {};
94
95     std::vector<HuffmanCode> sortedCodes = codes;
96     std::sort(sortedCodes.begin(), sortedCodes.end(), [](const HuffmanCode& a, const
97 HuffmanCode& b){
98         return a.c < b.c;
99     });
100     return sortedCodes;
101 }

```

Plik źródłowy 14: /Struktury/Huffman.cpp

4.3 Edge.h

Struktura *Edge* reprezentuje pojedynczą krawędź w grafie przepływowym. Przechowuje wierzchołek docelowy *to*, aktualny przepływ *flow*, przepustowość *capacity*, indeks krawędzi odwrotnej *rev* oraz koszt jednostkowy *cost*. Pole *rev* pozwala szybko odnaleźć krawędź przeciwną w grafie rezydualnym podczas aktualizacji przepływu.

```
1 #ifndef EDGE_H
2 #define EDGE_H
3
4 struct Edge {
5     int to;           // dokąd idzie
6     int flow;         // przepływ
7     int capacity;     // pojemność
8     int rev;          // ile węzłów ma dany node
9     int cost;         // ile posiłków musi zjeść krasnolud, żeby przebyć drogę
10
11     Edge(int to, int capacity, int rev, int cost) :
12         to(to), flow(0), capacity(capacity), rev(rev), cost(cost) {}
13 };
14
15 #endif
```

Plik źródłowy 15: /Struktury/Edge.h

4.4 Graph.cpp

Klasa *Graph* przechowuje graf jako listę sąsiedztwa. Metoda *add_edge(from, to, capacity, cost)* dodaje jednocześnie krawędź właściwą oraz krawędź odwrotną. Krawędź odwrotna ma początkową przepustowość 0 i koszt przeciwny do kosztu krawędzi właściwej. Dzięki temu algorytm minimalnego kosztu maksymalnego przepływu może cofać wcześniejsze decyzje i poprawiać przydział globalnie, a nie tylko zachłannie.

```
1 #ifndef GRAPH_H
2 #define GRAPH_H
3
4 #include <vector>
5 #include "Edge.h"
6
7 class Graph {
8 public:
9     int n; // ilość domów i kopalni
10     std::vector<std::vector<Edge>> graph;
11
12     Graph(int n) : n(n), graph(n) {}
13
14     void add_edge(int from, int to, int capacity, int cost);
15     void clear();
16 };
17
18 #endif
```

Plik źródłowy 16: /Struktury/Graph.h

```
1 #include "Graph.h"
2
3 void Graph::add_edge(int from, int to, int capacity, int cost) {
4     graph[from].push_back(Edge(to, capacity, (int)graph[to].size(), cost));
5     graph[to].push_back(Edge(from, 0, (int)graph[from].size() - 1, -cost));
6 }
7
8 void Graph::clear() {
9     for (auto& edge : graph) edge.clear();
10 }
```

Plik źródłowy 17: /Struktury/Graph.cpp

4.5 Kopalnia.cpp

Klasa *Kopalnia* opisuje jedno miejsce pracy. Przechowuje identyfikator, położenie, typ wydobywanego surowca, pojemność oraz listę identyfikatorów przydzielonych krasnali. Metody *getAvailableCapacity()*, *hasAvailableSpace()* oraz *isInUse()* pozwalają sprawdzić, czy kopalnia ma jeszcze wolne miejsca i czy została użyta w wyznaczonym przydziale. Metoda *addDwarf()* nie dodaje krasnała, jeżeli kopalnia jest pełna albo dany krasnał znajduje się już na liście.

```
1 #ifndef KOPALNIA_H
2 #define KOPALNIA_H
3
4 #include "Point.h"
5 #include <string>
6 #include <vector>
7 #include <algorithm>
8
9 class Kopalnia {
10 private:
11     int id;
12     Point location;
13     std::string resourceType;
14     int capacity;
15     std::vector<int> assignedDwarves;
16
17 public:
18     Kopalnia();
19
20     Kopalnia(int id, const Point& location, const std::string& resourceType,
21             int capacity, const std::vector<int>& assignedDwarves);
22
23     int getId() const;
24     const Point& getLocation() const;
25     double getX() const;
26     double getY() const;
27     const std::string& getResourceType() const;
28     int getCapacity() const;
29     const std::vector<int>& getAssignedDwarves() const;
30
31     void setId(int newId);
32     void setLocation(const Point& newLocation);
33     void setX(double newX);
34     void setY(double newY);
35     void setResourceType(const std::string& newResourceType);
36     void setCapacity(int newCapacity);
37     void setAssignedDwarves(const std::vector<int>& newAssignedDwarves);
38
39     int getAvailableCapacity() const;
40     bool hasAvailableSpace() const;
41     bool isInUse() const;
42     void addDwarf(int dwarfId);
43     void removeDwarf(int dwarfId);
44 };
45
46 #endif
```

Plik źródłowy 18: /Struktury/Kopalnia.h

```
1 #include "Kopalnia.h"
2
3 Kopalnia::Kopalnia()
4     : id(0), location(0.0, 0.0), resourceType(""), capacity(0), assignedDwarves({})
5 {}
6
7 Kopalnia::Kopalnia(int id, const Point& location, const std::string& resourceType,
8                     int capacity, const std::vector<int>& assignedDwarves)
9     : id(id),
10       location(location),
11       resourceType(resourceType),
```

```

11     capacity(capacity),
12     assignedDwarves(assignedDwarves) {}
13
14 int Kopalnia::getId() const {
15     return id;
16 }
17
18 const Point& Kopalnia::getLocation() const {
19     return location;
20 }
21
22 double Kopalnia::getX() const {
23     return location.x;
24 }
25
26 double Kopalnia::getY() const {
27     return location.y;
28 }
29
30 const std::string& Kopalnia::getResourceType() const {
31     return resourceType;
32 }
33
34 int Kopalnia::getCapacity() const {
35     return capacity;
36 }
37
38 const std::vector<int>& Kopalnia::getAssignedDwarves() const {
39     return assignedDwarves;
40 }
41
42 void Kopalnia::setId(int newId) {
43     this->id = newId;
44 }
45
46 void Kopalnia::setLocation(const Point& newLocation) {
47     this->location = newLocation;
48 }
49
50 void Kopalnia::setX(double newX) {
51     this->location.x = newX;
52 }
53
54 void Kopalnia::setY(double newY) {
55     this->location.y = newY;
56 }
57
58 void Kopalnia::setResourceType(const std::string& newResourceType) {
59     this->resourceType = newResourceType;
60 }
61
62 void Kopalnia::setCapacity(int newCapacity) {
63     this->capacity = newCapacity;
64 }
65
66 void Kopalnia::setAssignedDwarves(const std::vector<int>& newAssignedDwarves) {
67     this->assignedDwarves = newAssignedDwarves;
68 }
69
70 int Kopalnia::getAvailableCapacity() const {
71     return capacity - static_cast<int>(assignedDwarves.size());
72 }
73
74 bool Kopalnia::hasAvailableSpace() const {
75     return getAvailableCapacity() > 0;
76 }
77
78 bool Kopalnia::isInUse() const {

```

```

79     return !assignedDwarves.empty();
80 }
81
82 void Kopalnia::addDwarf(int dwarfId) {
83     if (!hasAvailableSpace()) {
84         return;
85     }
86
87     if (std::find(assignedDwarves.begin(), assignedDwarves.end(), dwarfId) !=
        assignedDwarves.end()) {
88         return;
89     }
90
91     assignedDwarves.push_back(dwarfId);
92 }
93
94 void Kopalnia::removeDwarf(int dwarfId) {
95     auto it = std::find(assignedDwarves.begin(), assignedDwarves.end(), dwarfId);
96
97     if (it != assignedDwarves.end()) {
98         assignedDwarves.erase(it);
99     }
100 }

```

Plik źródłowy 19: /Struktury/Kopalnia.cpp

4.6 Krasnal.cpp

Klasa *Krasnal* opisuje pracownika. Przechowuje identyfikator, położenie domu, listę umiejętności, preferowany surowiec oraz informację, czy ostatecznie został przydzielony do pracy zgodnej z preferencją. Umiejętności krasnala są później porównywane z surowcem kopalni, aby określić, czy dana krawędź krasnal–kopalnia może istnieć w grafie przydziału pracy.

```

1  #ifndef KRASNAL_H
2  #define KRASNAL_H
3
4  #include "Point.h"
5  #include <string>
6  #include <vector>
7
8  class Krasnal {
9  private:
10     int id;
11     Point home;
12     std::vector<std::string> skills;
13     std::string preferredResource;
14     bool assignedToPreferredResource;
15
16 public:
17     Krasnal();
18
19     Krasnal(int id, const Point& home,
20             const std::vector<std::string>& skills,
21             const std::string& preferredResource, bool assignedToPreferredResource);
22
23     int getId() const;
24     const Point& getHome() const;
25     double getHomeX() const;
26     double getHomeY() const;
27     const std::vector<std::string>& getSkills() const;
28     const std::string& getPreferredResource() const;
29     bool isAssignedToPreferredResource() const;
30
31     void setId(int newId);
32     void setHome(const Point& newHome);
33     void setHomeX(double newX);

```

```

34     void setHomeY(double newY);
35     void setSkills(const std::vector<std::string>& newSkills);
36     void setPreferredResource(const std::string& newPreferredResource);
37     void setAssignedToPreferredResource(bool assignedToPreferredResource);
38 };
39
40 #endif

```

Plik źródłowy 20: /Struktury/Krasnal.h

```

1  #include "Krasnal.h"
2
3  Krasnal::Krasnal()
4      : id(0), home(0.0, 0.0), skills({}), preferredResource(""),
      assignedToPreferredResource(false) {}
5
6  Krasnal::Krasnal(int id, const Point& home,
7                      const std::vector<std::string>& skills,
8                      const std::string& preferredResource,
9                      bool assignedToPreferredResource)
10      : id(id),
11        home(home),
12        skills(skills),
13        preferredResource(preferredResource),
14        assignedToPreferredResource(assignedToPreferredResource) {}
15
16  int Krasnal::getId() const{
17      return id;
18  }
19
20  const Point& Krasnal::getHome() const{
21      return home;
22  }
23
24  double Krasnal::getHomeY() const{
25      return home.y;
26  }
27
28  double Krasnal::getHomeX() const{
29      return home.x;
30  }
31
32  const std::vector<std::string>& Krasnal::getSkills() const{
33      return skills;
34  }
35
36  const std::string& Krasnal::getPreferredResource() const{
37      return preferredResource;
38  }
39
40  bool Krasnal::isAssignedToPreferredResource() const{
41      return assignedToPreferredResource;
42  }
43
44  void Krasnal::setId(int newId){
45      this->id = newId;
46  }
47
48  void Krasnal::setHome(const Point& newHome){
49      this->home = newHome;
50  }
51
52  void Krasnal::setHomeX(double newX){
53      this->home.x = newX;
54  }
55
56  void Krasnal::setHomeY(double newY){
57      this->home.y = newY;

```

```

58 }
59
60 void Krasnal::setSkills(const std::vector<std::string>& newSkills){
61     this->skills = newSkills;
62 }
63 void Krasnal::setPreferredResource(const std::string& newPreferredResource){
64     this->preferredResource = newPreferredResource;
65 }
66 void Krasnal::setAssignedToPreferredResource(bool assignedToPreferredResource){
67     this->assignedToPreferredResource = assignedToPreferredResource;
68 }

```

Plik źródłowy 21: /Struktury/Krasnal.cpp

4.7 MinCostMaxFlow.cpp

Funkcja `min_cost_max_flow()` wyznacza przepływ o możliwie najmniejszym koszcie. W projekcie służy do przydzielania krasnali do kopalni. Jedna jednostka przepływu odpowiada jednemu przydzielonemu krasnalowi.

Algorytm działa iteracyjnie. W każdej iteracji uruchamiany jest Bellman-Ford, który znajduje najtańszą dostępną ścieżkę od źródła do ujścia w grafie rezydualnym. Brane są pod uwagę tylko te krawędzie, dla których pozostała dodatnia przepustowość, czyli $capacity - flow > 0$. Dla każdego wierzchołka zapamiętywany jest poprzedni wierzchołek oraz indeks krawędzi, aby po znalezieniu ścieżki można było ją odtworzyć.

Po znalezieniu ścieżki obliczane jest jej wąskie gardło, czyli maksymalny przepływ, który można nią jeszcze przesłać. Następnie przepływ jest dodawany na krawędziach właściwych i odejmowany na krawędziach odwrotnych. Krawędzie odwrotne z ujemnym kosztem pozwalają algorytmowi cofnąć część wcześniejszych decyzji, jeżeli późniejsza konfiguracja okaże się tańsza.

Wynikiem jest struktura `MinCostMaxFlowResult`, która przechowuje łączną wartość przepływu oraz jego całkowity koszt.

Pewnego poranka do królestwa dotarła tajemnicza gazetka, według której w pobliskiej Biedronce obowiązywała promocja 12 + 12 na piwa. Wiadomość ta wywołała wśród krasnali tak duże poruszenie, że dotychczasowy problem minimalizacji drogi do kopalni musiał zostać chwilowo rozszerzony o koszt dojścia do alejki promocyjnej. Królowa Śnieżka nakazała zachować spokój i nie przekraczać budżetu, książę rozpoczął ponowne przeliczanie przepływu, a strażnicy graniczni otrzymali dodatkowe zadanie ochrony koszyków przed nieautoryzowaną zamianą miejsc w kolejce.

```

1  #ifndef MINCOSTMAXFLOW_H
2  #define MINCOSTMAXFLOW_H
3
4  #include "Graph.h"
5  #include <limits>
6
7  struct MinCostMaxFlowResult{
8      int flow;
9      long long cost;
10
11      MinCostMaxFlowResult() : flow(0), cost(0) {}
12      MinCostMaxFlowResult(int flow, long long cost) : flow(flow), cost(cost) {}
13 };
14
15 MinCostMaxFlowResult min_cost_max_flow(
16     Graph& graph,
17     int source,
18     int sink,
19     int maxFlow = std::numeric_limits<int>::max()
20 );
21
22 #endif

```

Plik źródłowy 22: /Struktury/MinCostMaxFlow.h

```

1  #include "MinCostMaxFlow.h"
2  #include <algorithm>
3  #include <limits>

```

```

4
5 using namespace std;
6
7 static bool bellman_ford(const Graph& graph, int source, int sink,
8     vector<long long>& dist, vector<int>& parentVertex, vector<int>& parentEdge) {
9
10     const long long INF = numeric_limits<long long>::max() / 4;
11
12     dist.assign(graph.n, INF);
13     //Wierzecholek
14     parentVertex.assign(graph.n, -1);
15     parentEdge.assign(graph.n, -1);
16     dist[source] = 0;
17
18     for ( int i = 0; i < graph.n - 1; i++ ) {
19
20         bool changed = false;
21
22         for ( int j = 0; j < graph.n; j++ ) {
23             if ( dist[j] == INF ) {
24                 continue;
25             }
26
27             for (size_t edgeIndex = 0; edgeIndex < graph.graph[j].size(); edgeIndex++) {
28                 const Edge& edge = graph.graph[j][edgeIndex];
29
30                 int residual = edge.capacity - edge.flow;
31                 if (residual <= 0) {
32                     continue;
33                 }
34
35                 long long candidate = dist[j] + edge.cost;
36                 if (candidate < dist[edge.to]) {
37                     dist[edge.to] = candidate;
38                     parentVertex[edge.to] = j;
39                     parentEdge[edge.to] = static_cast<int>(edgeIndex);
40                     changed = true;
41                 }
42             }
43         }
44         if (!changed) {
45             break;
46         }
47     }
48
49 }
50
51 return dist[sink] != INF;
52 }
53
54 }
55
56 MinCostMaxFlowResult min_cost_max_flow(Graph& graph, int source, int sink, int
57     maxFlow) {
58     MinCostMaxFlowResult result;
59
60     if ( source < 0 || source >= graph.n || sink < 0 || sink >= graph.n ) {
61         return result;
62     }
63
64     if ( source == sink || maxFlow <= 0 ) {
65         return result;
66     }
67
68     vector<long long> dist;
69     vector<int> parentVertex;
70     vector<int> parentEdge;

```



```

71 while (result.flow < maxFlow && bellman_ford(graph, source, sink, dist,
72     parentVertex, parentEdge)) {
73     int pushedFlow = maxFlow - result.flow;
74     for ( int vertex = sink; vertex != source; vertex = parentVertex[vertex]) {
75         int previousVertex = parentVertex[vertex];
76         int edgeIndex = parentEdge[vertex];
77         const Edge& edge = graph.graph[previousVertex][edgeIndex];
78
79         pushedFlow = min(pushedFlow, edge.capacity - edge.flow);
80     }
81
82     if (pushedFlow <= 0) {
83         break;
84     }
85     for ( int vertex = sink; vertex != source; vertex = parentVertex[vertex] ) {
86         int previousVertex = parentVertex[vertex];
87         int edgeIndex = parentEdge[vertex];
88         Edge& edge = graph.graph[previousVertex][edgeIndex];
89         Edge& reverseEdge = graph.graph[vertex][edge.rev];
90         edge.flow += pushedFlow;
91         reverseEdge.flow -= pushedFlow;
92     }
93     result.flow += pushedFlow;
94     result.cost += static_cast<long long>(pushedFlow) * dist[sink];
95 }
96 return result;
97 }
98 }

```

Plik źródłowy 23: /Struktury/MinCostMaxFlow.cpp

4.8 SegmentTree.cpp

Struktura danych jest drzewem przedziałowym, służącym do efektywnego wykonywania operacji wyszukiwania maksimum na zadanym przedziale tablicy oraz aktualizacji pojedynczych elementów.

4.8.1 build()

Proces budowy drzewa realizowany jest rekurencyjnie. Dla przedziału zawierającego jeden element wartość węzła ustawiana jest bezpośrednio na wartość elementu tablicy. W.p.p. przedział dzielony jest na dwie części. Następnie rekurencyjnie budowane są poddrzewa reprezentujące oba fragmenty.

4.8.2 query()

Umożliwia znalezienie największej wartości w zadanym przedziale indeksów. Algorytm analizuje relację pomiędzy aktualnym przedziałem reprezentowanym przez węzeł a zakresem zapytania. W przypadku braku części wspólnej zwracana jest najmniejsza możliwa wartość typu całkowitego. Jeżeli aktualny przedział w całości zawiera się w zakresie zapytania, zwracana jest wartość zapisana w danym węźle. W przeciwnym przypadku wykonywane są rekurencyjne zapytania dla lewego i prawego poddrzewa, a wyniki łączone są za pomocą operacji maksimum.

4.8.3 update()

Pozwala zmienić wartość pojedynczego elementu tablicy. Algorytm przechodzi od korzenia drzewa do odpowiedniego liścia reprezentującego aktualizowany indeks. Po zmianie wartości liścia następuje ponowne przeliczenie wartości wszystkich odwiedzonych wcześniej węzłów, dzięki czemu drzewo zachowuje poprawność przechowywanych danych

```

1 #include "SegmentTree.h"
2 #include <iostream>
3 using namespace std;

```

```

4
5 // !!! Indeksowanie od 0
6
7 void SegmentTree::build(int v, int l, int r)
8 {
9     if (l == r)
10     {
11         tree[v] = A[l];
12         return;
13     }
14
15     int mid = (l + r) / 2;
16     build(2 * v + 1, l, mid);
17     build(2 * v + 2, mid + 1, r);
18
19     tree[v] = max(tree[2 * v + 1], tree[2 * v + 2]);
20 }
21
22 int SegmentTree::query(int v, int l, int r, int ql, int qr)
23 {
24     if (r < ql || qr < l)
25         return numeric_limits<int>::lowest();
26     ; // rozlaczne
27     if (ql <= l && r <= qr)
28         return tree[v]; // zawarty
29
30     int mid = (l + r) / 2;
31     int x = query(2 * v + 1, l, mid, ql, qr);
32     int y = query(2 * v + 2, mid + 1, r, ql, qr);
33
34     return max(x, y);
35 }
36
37 void SegmentTree::update(int v, int l, int r, int idx, int x)
38 {
39     if (l == r)
40     {
41         tree[v] = x;
42         return;
43     }
44
45     int mid = (l + r) / 2;
46     if (idx <= mid)
47         update(2 * v + 1, l, mid, idx, x);
48     else
49         update(2 * v + 2, mid + 1, r, idx, x);
50
51     tree[v] = max(tree[2 * v + 1], tree[2 * v + 2]);
52 }
53
54 SegmentTree::SegmentTree(const vector<int> &a) : A(a), n(a.size())
55 {
56     tree.resize(4 * n);
57     build(0, 0, n - 1);
58 }
59
60 void SegmentTree::update(int i, int val)
61 {
62     update(0, 0, n - 1, i, val);
63 }
64
65 int SegmentTree::query(int l, int r)
66 {
67     return query(0, 0, n - 1, l, r);
68 }

```

Plik źródłowy 24: /Struktury/SegmentTree.cpp

4.9 Straznik.cpp

Klasa *Straznik* przechowuje identyfikator strażnika oraz jego głośność. Dane te są wykorzystywane przez *GuardCommandSolver*, który na podstawie głośności wybiera strażnika wydającego rozkaz na danym odcinku granicy.

```
1 #ifndef STRAZNIK_H
2 #define STRAZNIK_H
3
4 class Straznik {
5 private:
6     int id;
7     int loudness;
8
9 public:
10    Straznik();
11
12    Straznik(int id, int loudness);
13    int getId() const;
14    int getLoudness() const;
15
16    void setLoudness(int newLoudness);
17    void setId(int newId);
18 };
19
20 #endif
```

Plik źródłowy 25: /Struktury/Straznik.h

```
1 #include "Straznik.h"
2
3 Straznik::Straznik() : id(0), loudness(0) {}
4
5 Straznik::Straznik(int id, int loudness) : id(id), loudness(loudness) {}
6
7 int Straznik::getId() const {
8     return id;
9 }
10
11 int Straznik::getLoudness() const {
12     return loudness;
13 }
14
15 void Straznik::setLoudness(int newLoudness) {
16     this->loudness = newLoudness;
17 }
18
19 void Straznik::setId(int newId) {
20     this->id = newId;
21 }
```

Plik źródłowy 26: /Struktury/Straznik.cpp

5 Wyszukiwanie wzorca

5.1 Funkcje pomocnicze

- printFolderContents - wypisywanie zawartości folderu
- getFolderPath - wypisuje zawartość foldera i numeruje pliki
- getFile - wybieranie trybu wczytywania pliku i zwracanie ścieżki do tego pliku
- getPattern - pobiera wzorzec podane od użytkownika
- getFileContent - pobiera zawartość pliku

- breakString - dzieli tekst ze względu na separator
- writePositions - wypisuje liniijkę i miejsce wzorca w pliku

5.2 Jak działa

Program pobiera plik od użytkownika na 3 sposoby

1. Ścieżka do pliku
2. Wybieranie pliku w obecnym folderze
3. Ścieżka do folderu i następnie wybranie pliku z niego

Następnie pobiera od użytkownika wzorzec, czyta plik i zapisuje do zmiennej, dzieli zawartość ze względu na wiersze, szuka wzorzec osobno w każdej liniijce oraz jeżeli znalazł to wypisuje liniijkę oraz miejsce w liniijce (pierwsze miejsce to 0)

5.3 Kod źródłowy

```

1  #include "Struktury/Funkcje/KMP.h"
2  #include <iostream>
3  #include <filesystem>
4  #include <fstream>
5  #include <string>
6  #include <sstream>
7  using namespace std;
8  namespace fs = std::filesystem;
9
10 string currentPath = fs::current_path().string();
11
12 int printFolderContents(string path = currentPath){
13     int i = 0;
14     for (const auto& entry : fs::directory_iterator(path)){
15         if(!fs::is_regular_file(entry.path())) continue;
16         std::cout<<++i<<": " <<entry.path().filename()<<'\n';
17     }
18     return i;
19 }
20
21 string getFolderFilePath(int id, string path = currentPath){
22     int i = 0;
23     for (const auto& entry : fs::directory_iterator(path)){
24         if(!fs::is_regular_file(entry.path())) continue;
25         if (++i == id){
26             return entry.path().string();
27         }
28     }
29     return "";
30 }
31
32 string getFile(){
33     int mode = -1;
34
35     do{
36         cout<<"What mode do you want to use?"<<endl;
37         cout<<"1: Path to file"<<endl;
38         cout<<"2: Search for file in current directory"<<endl;
39         cout<<"3: Path to folder and select file"<<endl;
40         cout<<"Select: ";
41         cin>>mode;
42     } while(mode < 1 || mode > 3);
43
44     string path;
45     if(mode == 1){

```

```

46         do{
47             cout<<"Enter the path to the file: ";
48             cin>>path;
49         } while(!fs::exists(path) || !fs::is_regular_file(path));
50     } else if(mode == 2){
51         int max = printFolderContents();
52         int id = -1;
53
54         do{
55             cout<<"Enter the file number: ";
56             cin>>id;
57         } while(id < 1 || id > max);
58
59         path = getFolderFilePath(id);
60     } else if(mode == 3){
61         do{
62             cout<<"Enter the path to the folder: ";
63             cin>>path;
64         } while(!fs::exists(path) || !fs::is_directory(path));
65
66         int max = printFolderContents(path);
67         int id = -1;
68         do{
69             cout<<"Enter the file number: ";
70             cin>>id;
71         } while(id < 1 || id > max);
72
73         path = getFolderFilePath(id, path);
74     }
75     return path;
76 }
77
78 string getPattern(){
79     cout<<"Enter the pattern to search for: ";
80     string pattern;
81     cin>>pattern;
82     return pattern;
83 }
84
85 string getFileContent(string path){
86     string content;
87     ifstream file(path);
88     if(file.is_open()){
89         string line;
90         while(getline(file, line)){
91             content += line + "\n";
92         }
93         file.close();
94     }
95     return content;
96 }
97
98 vector<string> breakString(string text, char separator){
99     vector<string> lines;
100
101     stringstream ss(text);
102     string line;
103
104     while(getline(ss, line)){
105         lines.push_back(line);
106     }
107
108     return lines;
109 }
110
111 void writePositions(vector<string> lines, string pattern){
112     int lineNumber = 1;
113     int totalFound = 0;

```

```

114     for(const auto& eachLine : lines){
115         vector<int> positions = KMP(eachLine, pattern);
116         if(!positions.empty()){
117             cout<<"Pattern found at line: "<<lineNumber<<" at positions: ";
118             for(int pos : positions){
119                 cout<<pos<<" ";
120                 totalFound++;
121             }
122             cout<<endl;
123         }
124         lineNumber++;
125     }
126
127     if(totalFound == 0){
128         cout<<"No pattern found in the file."<<endl;
129     } else{
130         cout<<"Total occurrences found: "<<totalFound<<endl;
131     }
132 }
133
134 int main(){
135     string file = getFile();
136     if(file.empty()){
137         cout<<"Error in getting file"<<endl;
138         return -1;
139     }
140
141     string pattern = getPattern();
142     if(pattern.empty()){
143         cout<<"Error in getting pattern"<<endl;
144         return -1;
145     }
146
147     string fileContent = getFileContent(file);
148     vector<string> lines = breakString(fileContent, '\n');
149
150     writePositions(lines, pattern);
151
152     return 0;
153 }

```

Plik źródłowy 27: /finder.cpp

6 Kompresja i dekompresja

6.1 Funkcje pomocnicze

Podobnie jak w wyszukiwaniu wzorca

- modeSelector - wybranie trybu programu
- printFolderContents - wypisywanie zawartości folderu
- getFolderPath - wypisuje zawartość foldera i numeruje pliki
- getFile - wybieranie trybu wczytywania pliku i zwracanie ścieżki do tego pliku
- getFileContent - pobiera zawartość pliku

6.2 Część wspólna

Pierw użytkownik wybiera tryb jego działania, kompresja czy dekompresja. Następnie jest pytany, tak jak w wyszukiwaniu wzorca, jak chce podać plik.

1. Ścieżka do pliku

2. Wybieranie pliku w obecnym folderze
3. Ścieżka do folderu i następnie wybranie pliku z niego

6.3 Kompresja

Czytamy zawartość pliku, używając algorytmu KMP szukamy dla każdego znaku liczbę jego wystąpień (nie trzeba KMP, można by zachłannie). Tworzymy dla każdego znaku, który występuje przynajmniej raz, kody Huffmana. Tworzymy mapę znaku na jego kod. Zapisujemy binarnie przetłumaczony plik jako nazwaPliku.huff oraz tablicę kodów Huffmana do .nazwaPliku.huff

6.4 Dekompresja

Czytamy plik, odtwarzamy oryginalną nazwę, czytamy tabelę kodów Huffmana do tego pliku. Odczytujemy zawartość pliku. Używając tabeli Huffmana dla tego pliku tłumaczymy go i zapisujemy do pliku.

6.5 Kod źródłowy

```
1 #include "Struktury/Huffman.h"
2 #include "Struktury/Funkcje/KMP.h"
3 #include <iostream>
4 #include <filesystem>
5 #include <string>
6 #include <fstream>
7 #include <unordered_map>
8 namespace fs = std::filesystem;
9 using namespace std;
10
11 string currentPath = fs::current_path().string();
12
13 int modeSelector(){
14     int mode;
15     do{
16         cout<<"Select mode:"<<endl;
17         cout<<"1. Compress file"<<endl;
18         cout<<"2. Decompress file"<<endl;
19         cin>>mode;
20     } while(mode != 1 && mode != 2);
21     return mode;
22 }
23
24 int printFolderContents(string path = currentPath){
25     int i = 0;
26     for (const auto& entry : fs::directory_iterator(path)){
27         if(!fs::is_regular_file(entry.path())) continue;
28         std::cout<<++i<<": " <<entry.path().filename()<<'\n';
29     }
30     return i;
31 }
32
33 string getFolderFilePath(int id, string path = currentPath){
34     int i = 0;
35     for (const auto& entry : fs::directory_iterator(path)){
36         if(!fs::is_regular_file(entry.path())) continue;
37         if (++i == id){
38             return entry.path().string();
39         }
40     }
41     return "";
42 }
43
44 string getFile(){
45     int mode = -1;
```

```

46
47
48     cout<<"What mode do you want to use?"<<endl;
49     cout<<"1: Path to file"<<endl;
50     cout<<"2: Search for file in current directory"<<endl;
51     cout<<"3: Path to folder and select file"<<endl;
52     cout<<"Select: ";
53     cin>>mode;
54 } while(mode < 1 || mode > 3);
55
56 string path;
57 if(mode == 1){
58     do{
59         cout<<"Enter the path to the file: ";
60         cin>>path;
61     } while(!fs::exists(path) || !fs::is_regular_file(path));
62 } else if(mode == 2){
63     int max = printFolderContents();
64     int id = -1;
65
66     do{
67         cout<<"Enter the file number: ";
68         cin>>id;
69     } while(id < 1 || id > max);
70
71     path = getFolderFilePath(id);
72 } else if(mode == 3){
73     do{
74         cout<<"Enter the path to the folder: ";
75         cin>>path;
76     } while(!fs::exists(path) || !fs::is_directory(path));
77
78     int max = printFolderContents(path);
79     int id = -1;
80     do{
81         cout<<"Enter the file number: ";
82         cin>>id;
83     } while(id < 1 || id > max);
84
85     path = getFolderFilePath(id, path);
86 }
87 return path;
88 }
89
90 string getFileContent(string path){
91     string content;
92     ifstream file(path);
93     if(file.is_open()){
94         string line;
95         while(getline(file, line)){
96             content += line + "\n";
97         }
98         file.close();
99     }
100     return content;
101 }
102
103 int main(){
104     int mode = modeSelector();
105
106     string file = getFile();
107
108     if(mode == 1){
109         string content = getFileContent(file);
110
111         vector<char> C;
112         vector<int> F;
113         for(int i = 0; i<256; i++){

```



```

114         vector<int> kmp = KMP(content, string(1, (char)i));
115         if(kmp.size() > 0){
116             C.push_back((char)i);
117             F.push_back(kmp.size());
118         }
119     }
120
121     vector<HuffmanCode> codes = Huffman(C, F);
122
123     unordered_map<char, string> codeMap;
124     for(const auto &hc : codes){
125         codeMap[hc.c] = hc.code;
126     }
127
128     string bits;
129     bits.reserve(content.size() * 2);
130     for(char ch : content){
131         auto it = codeMap.find(ch);
132         if(it != codeMap.end()) bits += it->second;
133     }
134
135     vector<unsigned char> outBytes;
136     unsigned char current = 0;
137     int bitCount = 0;
138     for(char b : bits){
139         current = (current<<1) | (b == '1');
140         bitCount++;
141         if(bitCount == 8){
142             outBytes.push_back(current);
143             current = 0;
144             bitCount = 0;
145         }
146     }
147     int padding = 0;
148     if(bitCount > 0){
149         current <= (8 - bitCount);
150         outBytes.push_back(current);
151         padding = 8 - bitCount;
152     }
153
154     fs::path inPath(file);
155     string filename = inPath.filename().string();
156     fs::path outPath = inPath.parent_path() / (filename + ".huff");
157     fs::path tablePath = inPath.parent_path() / ( "." + filename + ".huff");
158
159     ofstream outFile(outPath, ios::binary);
160     if(!outFile){
161         cout<<"Failed to open output file: "<<outPath<<endl;
162         return 1;
163     }
164     unsigned char padByte = static_cast<unsigned char>(padding);
165     outFile.write(reinterpret_cast<char*>(&padByte), 1);
166     if(!outBytes.empty()){
167         outFile.write(reinterpret_cast<char*>(outBytes.data()), outBytes.size());
168     }
169     outFile.close();
170
171     ofstream tableFile(tablePath, ios::binary);
172     if(!tableFile){
173         cout<<"Failed to open table file: "<<tablePath<<endl;
174         return 1;
175     }
176     for(const auto &hc : codes){
177         int ascii = static_cast<unsigned char>(hc.c);
178         tableFile<<ascii<< ' ' <<hc.code<< '\n';
179     }
180     tableFile.close();
181

```

```

182     cout<<"Compressed file written to: "<<outPath<<endl;
183     cout<<"Code table written to: "<<tablePath<<endl;
184 } else if(mode == 2){
185     fs::path inPath(file);
186     string compFilename = inPath.filename().string();
187     string originalFilename = compFilename;
188     if(compFilename.size() > 5 && compFilename.substr(compFilename.size() -
189 5) == ".huff"){
190         originalFilename = compFilename.substr(0, compFilename.size() - 5);
191     }
192     fs::path tablePath = inPath.parent_path() / ( "." + originalFilename + ".
193 huff");
194     fs::path outPath = inPath.parent_path() / originalFilename;
195     if(!fs::exists(tablePath)){
196         cout<<"Code table not found: "<<tablePath<<endl;
197         return 1;
198     }
199
200     unordered_map<string, char> decodeMap;
201     ifstream tableFile(tablePath);
202     if(!tableFile){
203         cout<<"Failed to open table file: "<<tablePath<<endl;
204         return 1;
205     }
206     string line;
207     while(getline(tableFile, line)){
208         if(line.empty()) continue;
209         istreamstream iss(line);
210         int ascii;
211         string code;
212         if(!(iss >> ascii >> code)) continue;
213         decodeMap[code] = static_cast<char>(ascii);
214     }
215     tableFile.close();
216
217     ifstream inFile(inPath, ios::binary);
218     if(!inFile){
219         cout<<"Failed to open compressed file: "<<inPath<<endl;
220         return 1;
221     }
222     unsigned char padByte = 0;
223     inFile.read(reinterpret_cast<char*>(&padByte), 1);
224     if(!inFile){
225         cout<<"Invalid compressed file: "<<inPath<<endl;
226         return 1;
227     }
228     int padding = static_cast<int>(padByte);
229
230     vector<unsigned char> dataBytes((istreambuf_iterator<char>(inFile)),
231 istreambuf_iterator<char>());
232     inFile.close();
233
234     ofstream outFile(outPath, ios::binary);
235     if(!outFile){
236         cout<<"Failed to open output file: "<<outPath<<endl;
237         return 1;
238     }
239     string curCode;
240     for(size_t idx = 0; idx < dataBytes.size(); ++idx){
241         unsigned char byte = dataBytes[idx];
242         int bitsInThisByte = 8;
243         if(idx + 1 == dataBytes.size() && padding > 0) bitsInThisByte = 8 -
padding;
244         for(int b = 7; b >= 8 - bitsInThisByte; --b){
245             char bitChar = ((byte >> b) & 1) ? '1' : '0';

```

```

246         curCode.push_back(bitChar);
247         auto it = decodeMap.find(curCode);
248         if(it != decodeMap.end()){
249             outFile.put(it->second);
250             curCode.clear();
251         }
252     }
253 }
254 outFile.close();
255
256 cout<<"Decompressed file written to: "<<outPath<<endl;
257 }
258 }

```

Plik źródłowy 28: /compresser.cpp

7 Solvery

7.1 BorderPatrolSolver

Klasa *BorderPatrolSolver* spina problem wyznaczania trasy patrolu po aktywnych kopalniach. Na wejściu otrzymuje listę punktów reprezentujących położenia kopalni, w których faktycznie pracują krasnale. Następnie za pomocą funkcji *graham_scan()* wyznacza otoczkę wypukłą tych punktów. Długość patrolu jest liczona jako suma długości kolejnych krawędzi tej otoczki. Funkcja *distance()* zwraca kwadrat odległości, dlatego przy sumowaniu używany jest pierwiastek.

7.1.1 calculateConvexHull()

Metoda wyznacza i zapamiętuje otoczkę wypukłą aktywnych kopalni. Jeżeli wynik został już wcześniej policzony, zwracana jest zapamiętana otoczka.

7.1.2 calculatePatrolDistance()

Metoda zwraca długość zamkniętej trasy patrolu po punktach otoczki. Dla zera lub jednego punktu zwracana jest wartość 0, ponieważ nie istnieje rzeczywista trasa do okrążenia.

7.1.3 Kod źródłowy

```

1  #ifndef BORDER_PATROL_H
2  #define BORDER_PATROL_H
3
4  #include <vector>
5  #include "Point.h"
6  #include "Funkcje/graham_scan.h"
7  #include "Funkcje/distance.h"
8  #include <cmath>
9
10 class BorderPatrolSolver {
11 private:
12     std::vector<Point> activeMines;
13     std::vector<Point> hull;
14     bool calculated;
15
16 public:
17     BorderPatrolSolver(const std::vector<Point> &mines);
18
19     std::vector<Point> calculateConvexHull();
20     double calculatePatrolDistance();
21 };
22
23
24 #endif

```

Plik źródłowy 29: /Struktury/BorderPatrolSolver.h

```

1 #include "BorderPatrolSolver.h"
2
3 BorderPatrolSolver::BorderPatrolSolver(const std::vector<Point> &mines)
4     : activeMines(mines), calculated(false) {}
5
6 std::vector<Point> BorderPatrolSolver::calculateConvexHull() {
7     if(!calculated){
8         hull = graham_scan(activeMines);
9         calculated = true;
10    }
11    return hull;
12 }
13
14 double BorderPatrolSolver::calculatePatrolDistance(){
15     if(!calculated){
16         hull = graham_scan(activeMines);
17         calculated = true;
18    }
19
20    if(hull.size() < 2) return 0.0;
21
22    double dist = 0.0;
23
24    for(size_t i = 0; i < hull.size(); i++)
25    {
26        Point a = hull[i];
27        Point b = hull[(i + 1) % hull.size()];
28        dist += std::sqrt(distance(a,b)); // bo distance liczy kwadrat odleglosci
29    }
30    return dist;
31 }

```

Plik źródłowy 30: /Struktury/BorderPatrolSolver.cpp

7.2 WorkAssignmentSolver

Klasa *WorkAssignmentSolver* rozwiązuje problem przydziału krasnali do kopalni. Problem został zamodelowany jako sieć przepływu o minimalnym koszcie. Wierzchołki grafu odpowiadają źródłu, krasnalom, kopalniom oraz ujściu. Krawędź ze źródła do krasnala ma przepustowość 1, ponieważ jeden krasnal może dostać najwyżej jedną pracę. Krawędź z kopalni do ujścia ma przepustowość równą pojemności kopalni. Krawędź z krasnala do kopalni istnieje tylko wtedy, gdy krasnal posiada kompetencję wymaganą przez surowiec kopalni.

7.2.1 Model kosztu

Koszt krawędzi krasnal–kopalnia składa się z odległości domu krasnala od kopalni oraz ewentualnej kary za brak preferowanego surowca. Odległość jest liczona jako odległość euklidesowa i skalowana do liczby całkowitej, ponieważ *MinCostMaxFlow* operuje na kosztach typu *int*. Kara za brak preferencji jest większa niż maksymalna możliwa różnica sumy odległości, dzięki czemu solver najpierw preferuje pracę przy ulubionym surowcu, a dopiero potem minimalizuje dystans.

7.2.2 Wynik

Wynikiem działania solvera jest struktura *WorkAssignmentResult*, która zawiera listę przydziałów, listę nieprzydzielonych krasnali, liczbę przydziałów, liczbę przydziałów zgodnych z preferencjami oraz łączny dystans. Solver przechowuje też kopie kopalni i krasnali po aktualizacji, dzięki czemu kolejne moduły mogą korzystać z informacji o aktywnych kopalniach.

7.2.3 Kod źródłowy

```

1 #ifndef WORKASSIGNMENTSOLVER_H
2 #define WORKASSIGNMENTSOLVER_H
3

```

```

4 #include "../Kopalnia.h"
5 #include "../Krasnal.h"
6 #include "../MinCostMaxFlow.h"
7
8 #include <string>
9 #include <vector>
10
11 struct WorkAssignment {
12     int dwarfId;
13     int mineId;
14     std::string resourceType;
15     double distance;
16     bool preferredResource;
17
18     WorkAssignment(int dwarfId, int mineId, const std::string& resourceType,
19                   double distance, bool preferredResource);
20 };
21
22 struct WorkAssignmentResult {
23     std::vector<WorkAssignment> assignments;
24     std::vector<int> unassignedDwarfIds;
25     int assignedCount;
26     int preferredAssignedCount;
27     double totalDistance;
28     long long optimizationCost;
29
30     WorkAssignmentResult();
31     bool allDwarvesAssigned() const;
32 };
33
34 class WorkAssignmentSolver {
35 private:
36     std::vector<Krasnal> dwarves;
37     std::vector<Kopalnia> mines;
38
39     int sourceNode() const;
40     int sinkNode() const;
41     int dwarfNode(size_t dwarfIndex) const;
42     int mineNode(size_t mineIndex) const;
43
44     bool canWorkAtMine(const Krasnal& dwarf, const Kopalnia& mine) const;
45     bool isPreferredMine(const Krasnal& dwarf, const Kopalnia& mine) const;
46     double calculateTravelDistance(const Krasnal& dwarf, const Kopalnia& mine) const;
47     int calculateTravelCost(const Krasnal& dwarf, const Kopalnia& mine) const;
48     int calculatePreferencePenalty() const;
49
50     void clearAssignments();
51     void buildGraph(Graph& graph, int preferencePenalty) const;
52     void extractAssignments(const Graph& graph, WorkAssignmentResult& result);
53
54 public:
55     WorkAssignmentSolver(const std::vector<Krasnal>& dwarves,
56                         const std::vector<Kopalnia>& mines);
57
58     WorkAssignmentResult solve();
59
60     const std::vector<Krasnal>& getDwarves() const;
61     const std::vector<Kopalnia>& getMines() const;
62 };
63
64 #endif

```

Plik źródłowy 31: /Struktury/WorkAssignmentSolver/WorkAssignmentSolver.h

```

1 #include "WorkAssignmentSolver.h"
2
3 #include "../Funkcje/distance.h"
4

```

```

5 #include <algorithm>
6 #include <cmath>
7 #include <limits>
8
9 namespace {
10 const int DISTANCE_SCALE = 1000;
11 }
12
13 WorkAssignment::WorkAssignment(int dwarfId, int mineId, const std::string&
    resourceType,
14                                double distance, bool preferredResource)
15 : dwarfId(dwarfId),
16   mineId(mineId),
17   resourceType(resourceType),
18   distance(distance),
19   preferredResource(preferredResource) {}
20
21 WorkAssignmentResult::WorkAssignmentResult()
22 : assignedCount(0),
23   preferredAssignedCount(0),
24   totalDistance(0.0),
25   optimizationCost(0) {}
26
27 bool WorkAssignmentResult::allDwarvesAssigned() const {
28     return unassignedDwarfIds.empty();
29 }
30
31 WorkAssignmentSolver::WorkAssignmentSolver(const std::vector<Krasnal>& dwarves,
32                                             const std::vector<Kopalnia>& mines)
33 : dwarves(dwarves), mines(mines) {}
34
35 int WorkAssignmentSolver::sourceNode() const {
36     return 0;
37 }
38
39 int WorkAssignmentSolver::sinkNode() const {
40     return 1 + static_cast<int>(dwarves.size()) + static_cast<int>(mines.size());
41 }
42
43 int WorkAssignmentSolver::dwarfNode(size_t dwarfIndex) const {
44     return 1 + static_cast<int>(dwarfIndex);
45 }
46
47 int WorkAssignmentSolver::mineNode(size_t mineIndex) const {
48     return 1 + static_cast<int>(dwarves.size()) + static_cast<int>(mineIndex);
49 }
50
51 bool WorkAssignmentSolver::canWorkAtMine(const Krasnal& dwarf, const Kopalnia& mine)
52     const {
53     const std::vector<std::string>& skills = dwarf.getSkills();
54     return std::find(skills.begin(), skills.end(), mine.getResourceType()) != skills.
55         end();
56 }
57
58 bool WorkAssignmentSolver::isPreferredMine(const Krasnal& dwarf, const Kopalnia& mine)
59     const {
60     return dwarf.getPreferredResource() == mine.getResourceType();
61 }
62
63 double WorkAssignmentSolver::calculateTravelDistance(const Krasnal& dwarf, const
64     Kopalnia& mine) const {
65     return std::sqrt(distance(dwarf.getHome(), mine.getLocation()));
66 }
67
68 int WorkAssignmentSolver::calculateTravelCost(const Krasnal& dwarf, const Kopalnia&
69     mine) const {
70     return static_cast<int>(std::round(calculateTravelDistance(dwarf, mine) *
71         DISTANCE_SCALE));
72 }

```

```

66 }
67
68 int WorkAssignmentSolver::calculatePreferencePenalty() const {
69     int maxTravelCost = 0;
70
71     for (const Krasnal& dwarf : dwarves) {
72         for (const Kopalnia& mine : mines) {
73             if (canWorkAtMine(dwarf, mine)) {
74                 maxTravelCost = std::max(maxTravelCost, calculateTravelCost(dwarf,
75 mine));
76             }
77         }
78     }
79
80     if (maxTravelCost == 0) {
81         return 1;
82     }
83
84     const long long penalty = static_cast<long long>(maxTravelCost) *
85                             static_cast<long long>(dwarves.size()) + 1;
86
87     if (penalty > std::numeric_limits<int>::max()) {
88         return std::numeric_limits<int>::max();
89     }
90
91     return static_cast<int>(penalty);
92 }
93
94 void WorkAssignmentSolver::clearAssignments() {
95     for (Krasnal& dwarf : dwarves) {
96         dwarf.setAssignedToPreferredResource(false);
97     }
98
99     for (Kopalnia& mine : mines) {
100         mine.setAssignedDwarves(std::vector<int>{});
101     }
102 }
103
104 void WorkAssignmentSolver::buildGraph(Graph& graph, int preferencePenalty) const {
105     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
106         graph.add_edge(sourceNode(), dwarfNode(dwarfIndex), 1, 0);
107     }
108
109     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
110         for (size_t mineIndex = 0; mineIndex < mines.size(); mineIndex++) {
111             const Krasnal& dwarf = dwarves[dwarfIndex];
112             const Kopalnia& mine = mines[mineIndex];
113
114             if (!canWorkAtMine(dwarf, mine)) {
115                 continue;
116             }
117
118             long long cost = calculateTravelCost(dwarf, mine);
119             if (!isPreferredMine(dwarf, mine)) {
120                 cost += preferencePenalty;
121             }
122
123             if (cost > std::numeric_limits<int>::max()) {
124                 cost = std::numeric_limits<int>::max();
125             }
126
127             graph.add_edge(dwarfNode(dwarfIndex), mineNode(mineIndex), 1, static_cast
<int>(cost));
128         }
129     }
130
131     for (size_t mineIndex = 0; mineIndex < mines.size(); mineIndex++) {
132         graph.add_edge(mineNode(mineIndex), sinkNode(), mines[mineIndex].getCapacity

```

```

132     (), 0);
133 }
134
135 void WorkAssignmentSolver::extractAssignments(const Graph& graph,
136 WorkAssignmentResult& result) {
137     const int firstMineNode = mineNode(0);
138     const int lastMineNode = sinkNode() - 1;
139
140     std::vector<bool> assigned(dwarves.size(), false);
141
142     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
143         const int currentDwarfNode = dwarfNode(dwarfIndex);
144
145         for (const Edge& edge : graph.graph[currentDwarfNode]) {
146             if (edge.flow <= 0 || edge.to < firstMineNode || edge.to > lastMineNode)
147             {
148                 continue;
149             }
150
151             const size_t mineIndex = static_cast<size_t>(edge.to - firstMineNode);
152             const Krasnal& dwarf = dwarves[dwarfIndex];
153             const Kopalnia& mine = mines[mineIndex];
154             const bool preferred = isPreferredMine(dwarf, mine);
155             const double realDistance = calculateTravelDistance(dwarf, mine);
156
157             result.assignments.emplace_back(dwarf.getId(), mine.getId(), mine.
158 getResourceType(),
159                                     realDistance, preferred);
160
161             result.assignedCount++;
162             result.totalDistance += realDistance;
163
164             if (preferred) {
165                 result.preferredAssignedCount++;
166             }
167
168             dwarves[dwarfIndex].setAssignedToPreferredResource(preferred);
169             mines[mineIndex].addDwarf(dwarf.getId());
170             assigned[dwarfIndex] = true;
171             break;
172         }
173     }
174
175     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
176         if (!assigned[dwarfIndex]) {
177             result.unassignedDwarfIds.push_back(dwarves[dwarfIndex].getId());
178         }
179     }
180 }
181
182 WorkAssignmentResult WorkAssignmentSolver::solve() {
183     clearAssignments();
184
185     WorkAssignmentResult result;
186     if (dwarves.empty() || mines.empty()) {
187         for (const Krasnal& dwarf : dwarves) {
188             result.unassignedDwarfIds.push_back(dwarf.getId());
189         }
190         return result;
191     }
192
193     const int vertexCount = sinkNode() + 1;
194     Graph graph(vertexCount);
195     const int preferencePenalty = calculatePreferencePenalty();
196
197     buildGraph(graph, preferencePenalty);
198
199     const int maxFlow = static_cast<int>(dwarves.size());

```



```

196     MinCostMaxFlowResult flowResult = min_cost_max_flow(graph, sourceNode(), sinkNode
197     (), maxFlow);
198     result.optimizationCost = flowResult.cost;
199
200     extractAssignments(graph, result);
201
202     return result;
203 }
204
205 const std::vector<Krasnal>& WorkAssignmentSolver::getDwarves() const {
206     return dwarves;
207 }
208
209 const std::vector<Kopalnia>& WorkAssignmentSolver::getMines() const {
210     return mines;
211 }

```

Plik źródłowy 32: /Struktury/WorkAssignmentSolver/WorkAssignmentSolver.cpp

7.3 GuardCommandSolver

Klasa *GuardCommandSolver* wykorzystuje *SegmentTree* do znajdowania najgłośniejszego strażnika na zadanym odcinku granicy. Na wejściu otrzymuje listę strażników w kolejności ustawienia wzdłuż trasy patrolu. Zapytanie *findLoudestGuard(l,r)* zwraca identyfikator strażnika, jego indeks na trasie oraz głośność.

7.3.1 Rozstrzyganie remisów

Samo drzewo przedziałowe zwraca maksimum liczbowe, dlatego w solverze głośność jest kodowana razem z indeksem strażnika. Jeżeli kilku strażników ma taką samą głośność, wybierany jest pierwszy z nich na danym odcinku. Daje to deterministyczny wynik.

7.3.2 Aktualizacja

Metoda *updateLoudness(index,newLoudness)* zmienia głośność strażnika i aktualizuje odpowiadający liść drzewa. Pozwala to reagować na zmianę danych bez przebudowy całej struktury od zera.

7.3.3 Kod źródłowy

```

1  #ifndef GUARD_COMMAND_SOLVER_H
2  #define GUARD_COMMAND_SOLVER_H
3
4  #include "../SegmentTree.h"
5  #include "../Straznik.h"
6
7  #include <memory>
8  #include <vector>
9
10 struct GuardCommandResult {
11     int index;
12     int guardId;
13     int loudness;
14     bool found;
15
16     GuardCommandResult();
17     GuardCommandResult(int index, int guardId, int loudness);
18 };
19
20 class GuardCommandSolver {
21 private:
22     std::vector<Straznik> guards;
23     std::unique_ptr<SegmentTree> loudnessTree;
24
25     std::vector<int> buildEncodedLoudnessValues() const;

```

```

26     int encodeValue(size_t index, int loudness) const;
27     int decodeIndex(int encodedValue) const;
28     bool isValidRange(int left, int right) const;
29
30 public:
31     explicit GuardCommandSolver(const std::vector<Straznik>& guards);
32
33     GuardCommandResult findLoudestGuard(int left, int right) const;
34     bool updateLoudness(int index, int newLoudness);
35
36     int size() const;
37     const std::vector<Straznik>& getGuards() const;
38 };
39
40 #endif

```

Plik źródłowy 33: /Struktury/GuardCommandSolver/GuardCommandSolver.h

```

1  #include "GuardCommandSolver.h"
2
3  #include <limits>
4  #include <stdexcept>
5
6  GuardCommandResult::GuardCommandResult()
7      : index(-1), guardId(-1), loudness(0), found(false) {}
8
9  GuardCommandResult::GuardCommandResult(int index, int guardId, int loudness)
10     : index(index), guardId(guardId), loudness(loudness), found(true) {}
11
12  GuardCommandSolver::GuardCommandSolver(const std::vector<Straznik>& guards)
13     : guards(guards), loudnessTree(nullptr) {
14     if (!this->guards.empty()) {
15         loudnessTree.reset(new SegmentTree(buildEncodedLoudnessValues()));
16     }
17 }
18
19 std::vector<int> GuardCommandSolver::buildEncodedLoudnessValues() const {
20     std::vector<int> values;
21     values.reserve(guards.size());
22
23     for (size_t i = 0; i < guards.size(); i++) {
24         values.push_back(encodeValue(i, guards[i].getLoudness()));
25     }
26
27     return values;
28 }
29
30 int GuardCommandSolver::encodeValue(size_t index, int loudness) const {
31     if (loudness < 0 || index >= guards.size()) {
32         throw std::invalid_argument("Invalid guard loudness value");
33     }
34
35     const long long base = static_cast<long long>(guards.size()) + 1;
36     const long long tieBreaker = static_cast<long long>(guards.size()) - static_cast<
37     long long>(index);
38     const long long encoded = static_cast<long long>(loudness) * base + tieBreaker;
39
40     if (encoded > std::numeric_limits<int>::max()) {
41         throw std::overflow_error("Guard loudness value is too large");
42     }
43
44     return static_cast<int>(encoded);
45 }
46
47 int GuardCommandSolver::decodeIndex(int encodedValue) const {
48     if (guards.empty()) {
49         return -1;
50     }
51 }

```

```

50
51     const int base = static_cast<int>(guards.size()) + 1;
52     const int tieBreaker = encodedValue % base;
53     return static_cast<int>(guards.size()) - tieBreaker;
54 }
55
56 bool GuardCommandSolver::isValidRange(int left, int right) const {
57     return left >= 0 &&
58            right >= left &&
59            right < static_cast<int>(guards.size()) &&
60            loudnessTree != nullptr;
61 }
62
63 GuardCommandResult GuardCommandSolver::findLoudestGuard(int left, int right) const {
64     if (!isValidRange(left, right)) {
65         return GuardCommandResult();
66     }
67
68     const int encodedResult = loudnessTree->query(left, right);
69     const int index = decodeIndex(encodedResult);
70
71     if (index < left || index > right) {
72         return GuardCommandResult();
73     }
74
75     return GuardCommandResult(index, guards[index].getId(), guards[index].getLoudness());
76 }
77
78 bool GuardCommandSolver::updateLoudness(int index, int newLoudness) {
79     if (index < 0 ||
80         index >= static_cast<int>(guards.size()) ||
81         newLoudness < 0 ||
82         loudnessTree == nullptr) {
83         return false;
84     }
85
86     guards[index].setLoudness(newLoudness);
87     loudnessTree->update(index, encodeValue(static_cast<size_t>(index), newLoudness));
88
89     return true;
90 }
91
92 int GuardCommandSolver::size() const {
93     return static_cast<int>(guards.size());
94 }
95
96 const std::vector<Straznik>& GuardCommandSolver::getGuards() const {
97     return guards;
98 }

```

Plik źródłowy 34: /Struktury/GuardCommandSolver/GuardCommandSolver.cpp

8 Integracja programu

8.1 KingdomManager

Klasa *KingdomManager* odpowiada za wykonanie głównego scenariusza programu. Najpierw ustala ścieżkę do pliku wejściowego, następnie uruchamia *DataLoader*, wykonuje przydział pracy, wyznacza aktywne kopalnie, liczy trasę patrolu i przygotowuje solver zapytań dla strażników. Dzięki temu plik *main.cpp* pozostaje krótki i ogranicza się do utworzenia managera oraz wywołania metody *run()*.

8.1.1 processWorkAssignment()

Metoda wczytuje dane, tworzy *WorkAssignmentSolver*, uruchamia przydział pracy i wypisuje listę przydziałów, liczbę przydzielonych krasnali, liczbę przydziałów zgodnych z preferencjami oraz łączny dystans.

8.1.2 processBorderPatrol()

Metoda wybiera kopalnie, do których trafił przynajmniej jeden krasnal, a następnie przekazuje ich położenia do *BorderPatrolSolver*. Wynikiem jest otoczka wypukła aktywnych kopalni i długość trasy patrolu.

8.1.3 processGuardCommand()

Metoda tworzy *GuardCommandSolver* na podstawie listy strażników i wykonuje przykładowe zapytania: dla całej trasy oraz dla przykładowego odcinka ataku.

8.1.4 Kod źródłowy

```
1 #include "Struktury/Manager/KingdomManager.h"
2
3 int main() {
4     try {
5         KingdomManager manager;
6         manager.run();
7     }
8     catch (const std::exception& e) {
9         std::cerr << "Blad: " << e.what() << '\n';
10        return 1;
11    }
12
13    return 0;
14 }
```

Plik źródłowy 35: /main.cpp

```
1 #ifndef KINGDOMMANAGER_H
2 #define KINGDOMMANAGER_H
3
4 #include "../Parser/DataLoader.h"
5 #include "../Struktury/BorderPatrolSolver.h"
6 #include "../Struktury/GuardCommandSolver/GuardCommandSolver.h"
7 #include "../Struktury/WorkAssignmentSolver/WorkAssignmentSolver.h"
8 #include "../Struktury/Visualizer.h"
9 #include <algorithm>
10 #include <exception>
11 #include <fstream>
12 #include <iomanip>
13 #include <iostream>
14 #include <string>
15 #include <vector>
16
17 class KingdomManager {
18 private:
19     std::string inputPath;
20     DataLoader loader;
21     InputData data;
22
23     WorkAssignmentSolver solver;
24     WorkAssignmentResult result;
25
26     std::vector<Point> activeMineLocations;
27
28     BorderPatrolSolver patrolSolver;
```

```

29     std::vector<Point> patrolHull;
30     double patrolDistance;
31
32     GuardCommandSolver guardSolver;
33
34     void resolvePath();
35     void processWorkAssignment();
36     void processBorderPatrol();
37     void processGuardCommand();
38
39 public:
40     KingdomManager();
41     void run();
42 };
43
44 #endif

```

Plik źródłowy 36: /Struktury/Manager/KingdomManager.h

```

1  #include "KingdomManager.h"
2
3  KingdomManager::KingdomManager()
4      : inputPath("Dane/dane.txt"),
5        solver(data.dwarves, data.mines),
6        patrolSolver(activeMineLocations),
7        guardSolver(data.guards) {}
8
9  void KingdomManager::resolvePath() {
10     std::ifstream inputFile(inputPath);
11     if (!inputFile.good()) {
12         inputPath = "../Dane/dane.txt";
13     }
14 }
15
16 void KingdomManager::processWorkAssignment() {
17     data = loader.loadFromFile(inputPath);
18
19     std::cout << "Krasnale: " << data.dwarves.size() << '\n';
20     std::cout << "Kopalnie: " << data.mines.size() << '\n';
21     std::cout << "Straznicy: " << data.guards.size() << '\n';
22
23     solver = WorkAssignmentSolver(data.dwarves, data.mines);
24     result = solver.solve();
25
26     std::cout << "\nPrzydzial pracy:\n";
27     for (const WorkAssignment& assignment : result.assignments) {
28         std::cout << "Krasnal " << assignment.dwarfId
29                   << " -> kopalnia " << assignment.mineId
30                   << " (" << assignment.resourceType << "), dystans: "
31                   << std::fixed << std::setprecision(2) << assignment.distance;
32
33         if (assignment.preferredResource) {
34             std::cout << " [preferowany surowiec]";
35         }
36         std::cout << '\n';
37     }
38
39     if (!result.unassignedDwarfIds.empty()) {
40         std::cout << "\nBez przydzialu:";
41         for (int dwarfId : result.unassignedDwarfIds) {
42             std::cout << ' ' << dwarfId;
43         }
44         std::cout << '\n';
45     }
46
47     std::cout << "\nPrzydzielono: " << result.assignedCount << "/" << data.dwarves.size() << '\n';
48     std::cout << "Przydzialy preferowane: " << result.preferredAssignedCount << '\n';

```

```

49     std::cout << "Laczny dystans: " << std::fixed << std::setprecision(2) << result.
totalDistance << '\n';
50 }
51
52 void KingdomManager::processBorderPatrol() {
53     std::cout << "\nAktywne kopalnie:\n";
54     for (const Kopalnia& mine : solver.getMines()) {
55         if (!mine.isInUse()) {
56             continue;
57         }
58
59         activeMineLocations.push_back(mine.getLocation());
60         std::cout << "Kopalnia " << mine.getId()
61             << " (" << mine.getResourceType() << ") "
62             << mine.getLocation()
63             << ", krasnale:";
64
65         for (int dwarfId : mine.getAssignedDwarves()) {
66             std::cout << ' ' << dwarfId;
67         }
68         std::cout << '\n';
69     }
70
71     patrolSolver = BorderPatrolSolver(activeMineLocations);
72     patrolHull = patrolSolver.calculateConvexHull();
73     patrolDistance = patrolSolver.calculatePatrolDistance();
74
75     std::cout << "\nPunkty trasy patrolu:\n";
76     if (patrolHull.empty()) {
77         std::cout << "Brak aktywnych kopalni.\n";
78     } else {
79         for (const Point& point : patrolHull) {
80             std::cout << point << '\n';
81         }
82     }
83     std::cout << "Dlugosc trasy patrolu: " << std::fixed << std::setprecision(2) <<
patrolDistance << '\n';
84 }
85
86 void KingdomManager::processGuardCommand() {
87     guardSolver = GuardCommandSolver(data.guards);
88
89     std::cout << "\nObrona granicy:\n";
90     if (guardSolver.size() == 0) {
91         std::cout << "Brak straznikow.\n";
92     } else {
93         GuardCommandResult fullRange = guardSolver.findLoudestGuard(0, guardSolver.
size() - 1);
94         if (fullRange.found) {
95             std::cout << "Najglosniejszy na calej trasie: straznik "
96                 << fullRange.guardId
97                 << " na pozycji " << fullRange.index
98                 << ", glosnosc: " << fullRange.loudness << '\n';
99         }
100
101         if (guardSolver.size() >= 3) {
102             const int left = 1;
103             const int right = std::min(3, guardSolver.size() - 1);
104             GuardCommandResult attackedRange = guardSolver.findLoudestGuard(left,
right);
105
106             if (attackedRange.found) {
107                 std::cout << "Odcinek ataku [" << left << ", " << right << "]:
straznik "
108                     << attackedRange.guardId
109                     << " na pozycji " << attackedRange.index
110                     << ", glosnosc: " << attackedRange.loudness << '\n';
111             }

```

```

112     }
113 }
114 }
115
116 void KingdomManager::run() {
117     resolvePath();
118     processWorkAssignment();
119     processBorderPatrol();
120     processGuardCommand();
121
122     Visualizer visualizer(
123         solver.getDwarves(),
124         solver.getMines(),
125         data.guards,
126         result,
127         patrolHull,
128         patrolDistance,
129         guardSolver
130     );
131
132     visualizer.run();
133 }

```

Plik źródłowy 37: /Struktury/Manager/KingdomManager.cpp

9 Testy funkcji

9.1 test_kmp.cpp

Test funkcji *KMP* sprawdza, czy algorytm znajduje wszystkie wystąpienia wzorca w tekście. Dla przykładowego tekstu oczekiwane są dwa dopasowania wzorca.

```

1 #include "../KMP.h"
2 #include<iostream>
3
4 bool test1(){
5     std::string T = "ABCDBABCABBCDABCAB";
6     std::string P = "ABCAB";
7
8     const std::vector<int> expected = {5, 13};
9     const std::vector<int> result = KMP(T, P);
10
11     if(result.size() != expected.size()){
12         return false;
13     }
14
15     for(unsigned int i{}; i<result.size(); i++){
16         if(result[i] != expected[i]){
17             return false;
18         }
19     }
20     return true;
21 }
22
23 int main(){
24     std::cout<<"Testy dla funkcji KMP:"<<std::endl;
25     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (2 matches)"<<std::endl;
26
27     return 0;
28 }

```

Plik źródłowy 38: /Struktury/Funkcje/Testy/test_kmp.cpp

9.2 test_det.cpp

Sprawdzamy poprawność działania funkcji *det()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include "../det.h"
2 #include <iostream>
3
4 bool test1(){
5     Point p1(0, 0);
6     Point p2(3, 1);
7     Point p3(1, 3);
8
9     const double expected = 8;
10
11     return (det(p1, p2, p3) == expected);
12 }
13
14 bool test2(){
15     Point p1(0, 0);
16     Point p2(3, 1);
17     Point p3(4, -1);
18
19     const double expected = -7;
20
21     return (det(p1, p2, p3) == expected);
22 }
23
24 bool test3(){
25     Point p1(0, 0);
26     Point p2(3, 1);
27     Point p3(6, 2);
28
29     const double expected = 0;
30
31     return (det(p1, p2, p3) == expected);
32 }
33
34 bool test4(){
35     Point p1(0, 0);
36     Point p2(4, 2);
37     Point p3(2, 1);
38
39     const double expected = 0;
40
41     return (det(p1, p2, p3) == expected);
42 }
43
44 bool test5(){
45     Point p1(0, 0);
46     Point p2(4, 2);
47     Point p3(6, 3);
48
49     const double expected = 0;
50
51     return (det(p1, p2, p3) == expected);
52 }
53
54 bool test6(){
55     Point p1(0, 0);
56     Point p2(4, 2);
57     Point p3(3, 3);
58
59     const double expected = 6;
60
61     return (det(p1, p2, p3) == expected);
62 }
63
64 int main(){
65     std::cout<<"Testy dla funkcji det:"<<std::endl;
66     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (det = 8)"<<std::endl;
67     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (det = -7)"<<std::endl;
68     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (det = 0)"<<std::endl;

```



```

69     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (det = 0)"<<std::endl;
70     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (det = 0)"<<std::endl;
71     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (det = 6)"<<std::endl;
72
73     return 0;
74 }

```

Plik źródłowy 39: /Struktury/Funkcje/Testy/test_det.cpp

```

1  #!/bin/bash
2
3  NAME="test_det.exe"
4
5  g++ -Wall -o $NAME test_det.cpp ../det.cpp ../../Point.cpp
6  ./ $NAME
7  rm $NAME

```

Plik źródłowy 40: /Struktury/Funkcje/Testy/run_test_det.sh

9.3 test_is_point_on_segment.cpp

Sprawdzamy poprawność działania funkcji *is_point_on_segment()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1  #include <iostream>
2  #include "../is_point_on_segment.cpp"
3
4  bool test1(){
5      Point p1(0,0), p2(4,2), p3(2,1);
6
7      const bool expected = true;
8
9      return (is_point_on_segment(p1,p2,p3) == expected);
10 }
11
12 bool test2(){
13     Point p1(0,0), p2(4,2), p4(6,3);
14
15     const bool expected = false;
16
17     return (is_point_on_segment(p1,p2,p4) == expected);
18 }
19
20 int main(){
21     std::cout<<"Testy dla funkcji is_point_on_segment:"<<std::endl;
22     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (on segment)"<<std::endl;
23     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (not on segment)"<<std::endl;
24
25     return 0;
26 }

```

Plik źródłowy 41: /Struktury/Funkcje/Testy/test_is_point_on_segment.cpp

```

1  #!/bin/bash
2
3  NAME="test_is_point_on_segment.exe"
4
5  g++ -Wall -o $NAME test_is_point_on_segment.cpp ../../Point.cpp ../det.cpp
6  ./ $NAME
7  rm $NAME

```

Plik źródłowy 42: /Struktury/Funkcje/Testy/run_test_is_point_on_segment.sh

9.4 test_do_segments_intersect.cpp

Sprawdzamy poprawność działania funkcji `do_segments_intersect()` wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```
1 #include <iostream>
2 #include "../do_segments_intersect.cpp"
3
4 // Odcinki rozłączne, nie przecinają się
5 bool test1(){
6     Point A(1, 1), B(4, 2), C(2, 4), D(5, 5);
7     const bool expected = false;
8
9     return (do_segments_intersect(A, B, C, D) == expected);
10 }
11
12 // Odcinki przecinają się klasycznie X
13 bool test2()
14 {
15     Point A(0,0), B(4,4), C(0,4), D(4,0);
16     return(do_segments_intersect(A,B,C,D) == true);
17 }
18
19
20 // Odcinki stykają się w jednym końcowym punkcie
21 bool test3()
22 {
23     Point A(0,0), B(2,2), C(2,2), D(4,0);
24     return(do_segments_intersect(A,B,C,D) == true);
25 }
26
27 // Odcinki współliniowe i nachodzące na siebie
28 bool test4()
29 {
30     Point A(0,0), B(4,0), C(2,0), D(6,0);
31     return(do_segments_intersect(A,B,C,D) == true);
32 }
33
34 // Odcinki współliniowe ale rozłączne
35 bool test5()
36 {
37     Point A(0,0), B(2,0), C(3,0), D(5,0);
38     return(do_segments_intersect(A,B,C,D) == false);
39 }
40
41 int main(){
42     std::cout<<"Testy dla funkcji do_segments_intersect:"<<std::endl;
43     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (do not intersect)"<<std::endl;
44     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (intersect)"<<std::endl;
45     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (touch at endpoint)"<<std::endl;
46     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (collinear overlapping)"<<std::endl;
47     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (collinear disjoining)"<<std::endl;
48     ; // współliniowe nachodzące
49     ; // współliniowe rozłączne
50
51     return 0;
52 }
```

Plik źródłowy 43: /Struktury/Funkcje/Testy/test_do_segments_intersect.cpp

```
1 #!/bin/bash
2
3 NAME="test_do_segments_intersect.exe"
4
5 g++ -Wall -o $NAME test_do_segments_intersect.cpp ../../Point.cpp ../../det.cpp ../../
6     is_point_on_segment.cpp
7 ./ $NAME
```

```
7 rm $NAME
```

Plik źródłowy 44: /Struktury/Funkcje/Testy/run_test_do_segments_intersect.sh

9.5 test_is_point_in_polygon.cpp

Sprawdzamy poprawność działania funkcji *is_point_in_polygon()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```
1 #include "../is_point_in_polygon.h"
2 #include <iostream>
3
4 bool test1(){
5     std::vector<Point> polygon = {Point(0, 0), Point(5, 1), Point(3, 3), Point(5, 5),
6     Point(2, 4), Point(0, 5)};
7     Point p = Point(3, 2);
8
9     const bool expected = true;
10
11     return (is_point_in_polygon(p, polygon) == expected);
12 }
13
14 bool test2(){
15     std::vector<Point> polygon = {Point(0, 0), Point(5, 1), Point(3, 3), Point(5, 5),
16     Point(2, 4), Point(0, 5)};
17     Point p = Point(4, 3);
18
19     const bool expected = false;
20
21     return (is_point_in_polygon(p, polygon) == expected);
22 }
23
24 int main(){
25     std::cout<<"Testy dla funkcji is_point_in_polygon:"<<std::endl;
26     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (in polygon)"<<std::endl;
27     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (not in polygon)"<<std::endl;
28
29     return 0;
30 }
```

Plik źródłowy 45: /Struktury/Funkcje/Testy/test_is_point_in_polygon.cpp

```
1 #!/bin/bash
2
3 NAME="test_is_point_in_polygon.exe"
4
5 g++ -Wall -o $NAME test_is_point_in_polygon.cpp ../is_point_in_polygon.cpp ../../
6 Point.cpp ../is_point_on_segment.cpp ../det.cpp ../do_segments_intersect.cpp
7 rm $NAME
```

Plik źródłowy 46: /Struktury/Funkcje/Testy/run_test_is_point_in_polygon.sh

9.6 test_graham_scan.cpp

Sprawdzamy poprawność działania funkcji *graham_scan()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```
1 #include "../graham_scan.h"
2 #include "../distance.h"
3 #include <iostream>
4
5 // Test 1 - pusyt zbior -> otoczka tez pusta
6 bool test1(){
7     std::vector<Point> points, expected;
8 }
```

```

9     points = {};
10
11     expected = {};
12     points = graham_scan(points);
13
14     if(points.size() != expected.size()) return false;
15
16     for(unsigned int i = 0; i < points.size(); i++){
17         if(points[i] != expected[i]) return false;
18     }
19
20     return true;
21 }
22
23 // Test 2 - jeden punkt
24 bool test2(){
25     std::vector<Point> points, expected;
26
27     points = {Point(0, 0)};
28
29     expected = points;
30     points = graham_scan(points);
31
32     if(points.size() != expected.size()) return false;
33
34     for(unsigned int i = 0; i < points.size(); i++){
35         if(points[i] != expected[i]) return false;
36     }
37
38     return true;
39 }
40
41 // Test 3 - dwa identyczne pkt -> jeden w otoczce
42 bool test3(){
43     std::vector<Point> points, expected;
44
45     points = {Point(0, 0), Point(0, 0)};
46
47     expected = {Point(0, 0)};
48     points = graham_scan(points);
49
50     if(points.size() != expected.size()) return false;
51
52     for(unsigned int i = 0; i < points.size(); i++){
53         if(points[i] != expected[i]) return false;
54     }
55
56     return true;
57 }
58
59 // Test 4 dwa rozne pkt + duplikat -> tylko unikalne pkt
60 bool test4(){
61     std::vector<Point> points, expected;
62
63     points = {Point(0, 0), Point(0, 0), Point(1, 2)};
64
65     expected = {Point(0, 0), Point(1, 2)};
66     points = graham_scan(points);
67
68     if(points.size() != expected.size()) return false;
69
70     for(unsigned int i = 0; i < points.size(); i++){
71         if(points[i] != expected[i]) return false;
72     }
73
74     return true;
75 }
76

```

```

77 // Test 5 - sprawdzenie poprawnego wyboru pkt
78 bool test5(){
79     std::vector<Point> points, expected;
80
81     points = {Point(0, 0), Point(1, 1), Point(2, 0), Point(3, 0)};
82
83     expected = {Point(0, 0), Point(3, 0), Point(1, 1)};
84     points = graham_scan(points);
85
86     if(points.size() != expected.size()) return false;
87
88     for(unsigned int i = 0; i < points.size(); i++){
89         if(points[i] != expected[i]) return false;
90     }
91
92     return true;
93 }
94
95 // Test 6 - poprawnosc pkt (wiekszy zbior)
96 bool test6(){
97     std::vector<Point> points, expected;
98
99     points = {
100         Point(0, 0), Point(1, 0), Point(2, 0), Point(2, 1), Point(4, 2),
101         Point(3, 4), Point(1, 3), Point(5, 1), Point(3, 0), Point(2, 2)
102     };
103
104     expected = {Point(0, 0), Point(3, 0), Point(5, 1), Point(3, 4), Point(1, 3)};
105     points = graham_scan(points);
106
107     if(points.size() != expected.size()) return false;
108
109     for(unsigned int i = 0; i < points.size(); i++){
110         if(points[i] != expected[i]) return false;
111     }
112
113     return true;
114 }
115
116 // Test 7 - punkty wspolliniowe -> tylko skrajne powinny zostac
117 bool test7()
118 {
119     std::vector<Point> points, expected;
120
121     points = {
122         Point(0,0), Point(1,1), Point(2,2),
123         Point(3,3), Point(4,4)
124     };
125
126     expected = {Point(0,0), Point(4,4)};
127     points = graham_scan(points);
128
129     if(points.size() != expected.size()) return false;
130
131     for(unsigned int i = 0; i < points.size(); i++){
132         if(points[i] != expected[i]) return false;
133     }
134
135     return true;
136 }
137
138 int main(){
139     std::cout<<"Testy dla funkcji graham_scan:"<<std::endl;
140     std::cout<<"Test 1: "<<(test1())?"OK":"ERROR"<<" (empty set)"<<std::endl;
141     std::cout<<"Test 2: "<<(test2())?"OK":"ERROR"<<" (1 element set)"<<std::endl;
142     std::cout<<"Test 3: "<<(test3())?"OK":"ERROR"<<" (2 element set)"<<std::endl;
143     std::cout<<"Test 4: "<<(test4())?"OK":"ERROR"<<" (3 element set)"<<std::endl;
144     std::cout<<"Test 5: "<<(test5())?"OK":"ERROR"<<" (4 element set)"<<std::endl;

```

```

145     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (10 element set)"<<std::endl;
146     std::cout<<"Test 7: "<<(test7()?"OK":"ERROR")<<" (collinear points)"<<std::endl;
147
148
149     return 0;
150 }

```

Plik źródłowy 47: /Struktury/Funkcje/Testy/test_graham_scan.cpp

```

1 #!/bin/bash
2
3 NAME="test_graham_scan.exe"
4
5 g++ -Wall -o $NAME test_graham_scan.cpp ../graham_scan.cpp ../../Point.cpp ../det.cpp
6     ../distance.cpp
7 ./ $NAME
8 rm $NAME

```

Plik źródłowy 48: /Struktury/Funkcje/Testy/run_test_graham_scan.sh

10 Testy struktur

10.1 test_data_loader.cpp

Testy *DataLoader* sprawdzają poprawne wczytywanie liczby obiektów, pełne dane pojedynczego krasnala, kopalni i strażnika, obsługę braku pliku oraz błędów formatu. Osobne przypadki testują ujemną liczbę umiejętności, ujemną pojemność kopalni i ujemną głośność strażnika.

```

1 #include "../DataLoader.h"
2 #include<iostream>
3 #include<fstream>
4 #include<cstdio>
5 #include<cmath>
6 #include<vector>
7
8 bool createFile(const std::string& fileName, const std::string& content){
9     std::ofstream file(fileName.c_str());
10
11     if(!file.is_open()){
12         return false;
13     }
14
15     file<<content;
16     return true;
17 }
18
19 bool compareDouble(double a, double b){
20     return std::fabs(a - b) < 0.000000001;
21 }
22
23 bool test1(){
24     const std::string fileName = "test_data_loader_1.txt";
25
26     if(!createFile(fileName,
27         "KRASNALE\n"
28         "1 0 0 zloto 2 zloto miedz\n"
29         "2 8 1 wegciel 1 wegciel\n"
30         "\n"
31         "KOPALNIE\n"
32         "10 2 2 zloto 1\n"
33         "11 9 2 wegciel 2\n"
34         "\n"
35         "STRAZNICY\n"
36         "100 14\n"
37         "101 9\n")){
38         return false;

```

```

39     }
40
41     try{
42         DataLoader loader;
43         InputData data = loader.loadFromFile(fileName);
44         std::remove(fileName.c_str());
45
46         return data.dwarves.size() == 2 &&
47                data.mines.size() == 2 &&
48                data.guards.size() == 2;
49     } catch(...){
50         std::remove(fileName.c_str());
51         return false;
52     }
53 }
54
55 bool test2(){
56     const std::string fileName = "test_data_loader_2.txt";
57
58     if(!createFile(fileName,
59                    "KRASNALE\n"
60                    "1 -3.5 4.25 zloto 2 zloto miedz\n")){
61         return false;
62     }
63
64     try{
65         DataLoader loader;
66         InputData data = loader.loadFromFile(fileName);
67         std::remove(fileName.c_str());
68
69         if(data.dwarves.size() != 1){
70             return false;
71         }
72
73         Krasnal dwarf = data.dwarves[0];
74         const std::vector<std::string> expectedSkills = {"zloto", "miedz"};
75
76         return dwarf.getId() == 1 &&
77                compareDouble(dwarf.getHomeX(), -3.5) &&
78                compareDouble(dwarf.getHomeY(), 4.25) &&
79                dwarf.getPreferredResource() == "zloto" &&
80                dwarf.getSkills() == expectedSkills &&
81                dwarf.isAssignedToPreferredResource() == false;
82     } catch(...){
83         std::remove(fileName.c_str());
84         return false;
85     }
86 }
87
88 bool test3(){
89     const std::string fileName = "test_data_loader_3.txt";
90
91     if(!createFile(fileName,
92                    "KOPALNIE\n"
93                    "10 2.5 -1.25 zloto 3\n")){
94         return false;
95     }
96
97     try{
98         DataLoader loader;
99         InputData data = loader.loadFromFile(fileName);
100        std::remove(fileName.c_str());
101
102        if(data.mines.size() != 1){
103            return false;
104        }
105
106        Kopalnia mine = data.mines[0];

```

```

107
108         return mine.getId() == 10 &&
109                compareDouble(mine.getX(), 2.5) &&
110                compareDouble(mine.getY(), -1.25) &&
111                mine.getResourceType() == "zloto" &&
112                mine.getCapacity() == 3 &&
113                mine.getAssignedDwarves().empty();
114     } catch(...){
115         std::remove(fileName.c_str());
116         return false;
117     }
118 }
119
120 bool test4(){
121     const std::string fileName = "test_data_loader_4.txt";
122
123     if(!createFile(fileName,
124                    "STRAZNICY\n"
125                    "100 14\n")){
126         return false;
127     }
128
129     try{
130         DataLoader loader;
131         InputData data = loader.loadFromFile(fileName);
132         std::remove(fileName.c_str());
133
134         if(data.guards.size() != 1){
135             return false;
136         }
137
138         Straznik guard = data.guards[0];
139
140         return guard.getId() == 100 &&
141                guard.getLoudness() == 14;
142     } catch(...){
143         std::remove(fileName.c_str());
144         return false;
145     }
146 }
147
148 bool test5(){
149     try{
150         DataLoader loader;
151         loader.loadFromFile("plik_ktory_nie_istnieje.txt");
152     } catch(const std::runtime_error& e){
153         return std::string(e.what()) == "Failed to open file";
154     } catch(...){
155         return false;
156     }
157
158     return false;
159 }
160
161 bool test6(){
162     const std::string fileName = "test_data_loader_6.txt";
163
164     if(!createFile(fileName,
165                    "KRASNALE\n"
166                    "1 0 0 zloto 2 zloto\n")){
167         return false;
168     }
169
170     try{
171         DataLoader loader;
172         loader.loadFromFile(fileName);
173     } catch(const std::runtime_error& e){
174         std::remove(fileName.c_str());

```



```

175         return std::string(e.what()) == "Failed to parse line";
176     } catch(...){
177         std::remove(fileName.c_str());
178         return false;
179     }
180
181     std::remove(fileName.c_str());
182     return false;
183 }
184
185 bool test7(){
186     const std::string fileName = "test_data_loader_7.txt";
187
188     if(!createFile(fileName,
189         "KRASNALE\n"
190         "1 0 0 zloto -1\n")){
191         return false;
192     }
193
194     try{
195         DataLoader loader;
196         loader.loadFromFile(fileName);
197     } catch(const std::runtime_error& e){
198         std::remove(fileName.c_str());
199         return std::string(e.what()) == "Negative skill count";
200     } catch(...){
201         std::remove(fileName.c_str());
202         return false;
203     }
204
205     std::remove(fileName.c_str());
206     return false;
207 }
208
209 bool test8(){
210     const std::string fileName = "test_data_loader_8.txt";
211
212     if(!createFile(fileName,
213         "KOPALNIE\n"
214         "10 2 2 zloto -1\n")){
215         return false;
216     }
217
218     try{
219         DataLoader loader;
220         loader.loadFromFile(fileName);
221     } catch(const std::runtime_error& e){
222         std::remove(fileName.c_str());
223         return std::string(e.what()) == "Negative capacity";
224     } catch(...){
225         std::remove(fileName.c_str());
226         return false;
227     }
228
229     std::remove(fileName.c_str());
230     return false;
231 }
232
233 bool test9(){
234     const std::string fileName = "test_data_loader_9.txt";
235
236     if(!createFile(fileName,
237         "STRAZNICY\n"
238         "100 -1\n")){
239         return false;
240     }
241
242     try{

```

```

243     DataLoader loader;
244     loader.loadFromFile(fileName);
245 } catch(const std::runtime_error& e){
246     std::remove(fileName.c_str());
247     return std::string(e.what()) == "Negative loudness";
248 } catch(...){
249     std::remove(fileName.c_str());
250     return false;
251 }
252
253 std::remove(fileName.c_str());
254 return false;
255 }
256
257 int main(){
258     std::cout<<"Testy dla klasy DataLoader:"<<std::endl;
259     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (liczba wczytanych danych)"<<std
::endl;
260     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (wczytanie krasnala)"<<std::endl
;
261     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (wczytanie kopalni)"<<std::endl;
262     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (wczytanie straznika)"<<std::
endl;
263     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (brak pliku)"<<std::endl;
264     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (bledna linia)"<<std::endl;
265     std::cout<<"Test 7: "<<(test7()?"OK":"ERROR")<<" (ujemna liczba kompetencji)"<<
std::endl;
266     std::cout<<"Test 8: "<<(test8()?"OK":"ERROR")<<" (ujemna pojemnosc)"<<std::endl;
267     std::cout<<"Test 9: "<<(test9()?"OK":"ERROR")<<" (ujemna glosnosc)"<<std::endl;
268
269     return 0;
270 }

```

Plik źródłowy 49: /Parser/Testy/test_data_loader.cpp

10.2 test_min_cost_max_flow.cpp

Testy *MinCostMaxFlow* sprawdzają wybór najtańszej ścieżki dla przepływu równego 1, poprawne wysłanie przepływu przez dwie ścieżki oraz sytuację, w której nie istnieje ścieżka ze źródła do ujścia.

```

1 #include "../Graph.h"
2 #include "../MinCostMaxFlow.h"
3
4 #include <iostream>
5
6 bool test1() {
7     Graph graph(4);
8
9     graph.add_edge(0, 1, 1, 2);
10    graph.add_edge(0, 2, 1, 1);
11    graph.add_edge(1, 3, 1, 2);
12    graph.add_edge(2, 3, 1, 5);
13
14    MinCostMaxFlowResult result = min_cost_max_flow(graph, 0, 3, 1);
15
16    return result.flow == 1 && result.cost == 4;
17 }
18
19 bool test2() {
20     Graph graph(4);
21
22     graph.add_edge(0, 1, 1, 2);
23     graph.add_edge(0, 2, 1, 1);
24     graph.add_edge(1, 3, 1, 2);
25     graph.add_edge(2, 3, 1, 5);
26
27     MinCostMaxFlowResult result = min_cost_max_flow(graph, 0, 3, 2);

```

```

28
29     return result.flow == 2 && result.cost == 10;
30 }
31
32 bool test3() {
33     Graph graph(4);
34
35     graph.add_edge(0, 1, 1, 2);
36     graph.add_edge(1, 2, 1, 3);
37
38     MinCostMaxFlowResult result = min_cost_max_flow(graph, 0, 3, 1);
39
40     return result.flow == 0 && result.cost == 0;
41 }
42
43 int main() {
44     std::cout << "Testy dla MinCostMaxFlow:" << std::endl;
45     std::cout << "Test 1: " << (test1() ? "OK" : "ERROR") << " (najtansza sciezka dla
46         maxFlow = 1)" << std::endl;
47     std::cout << "Test 2: " << (test2() ? "OK" : "ERROR") << " (dwie sciezki dla
48         maxFlow = 2)" << std::endl;
49     std::cout << "Test 3: " << (test3() ? "OK" : "ERROR") << " (brak sciezki do sink)
50         " << std::endl;
51
52     return 0;
53 }

```

Plik źródłowy 50: /Struktury/Funkcje/Testy/test_min_cost_max_flow.cpp

10.3 test_point.cpp

Sprawdzamy poprawność struktury *Point* poprzez sprawdzenie czy określone operatory poprawnie działają i wykonują to co chcemy

```

1 #include <iostream>
2 #include <sstream>
3 #include "../Point.h"
4
5 bool test1(){
6     Point A(1,2);
7     Point B(4,5);
8
9     const bool expected = false;
10
11     return ((A > B) == expected);
12 }
13
14 bool test2(){
15     Point A(1,2);
16     Point B(4,5);
17
18     const bool expected = true;
19
20     return ((A < B) == expected);
21 }
22
23 bool test3(){
24     Point A(1, 2);
25
26     try{
27         std::ostringstream str;
28         str<<A;
29         return true;
30     } catch(...){
31         return false;
32     }
33 }

```

```

34
35 bool test4(){
36     Point A(1, 2);
37     Point B(1, 2);
38
39     const bool expected = true;
40
41     return ((A == B) == expected);
42 }
43
44 bool test5(){
45     Point A(1, 2);
46     Point B(3, 4);
47
48     const bool expected = true;
49
50     return ((A != B) == expected);
51 }
52
53 bool test6(){
54     Point A(1, 2);
55     Point B(1, 2);
56
57     const bool expected = true;
58
59     return ((A <= B) == expected);
60 }
61
62 bool test7(){
63     Point A(1, 2);
64     Point B(0, 0);
65
66     const bool expected = true;
67
68     return ((A >= B) == expected);
69 }
70
71 bool test8(){
72     Point A(1, 2);
73     Point B(3, 4);
74     B = A;
75
76     const bool expected = true;
77
78     return ((A == B) == expected);
79 }
80
81 int main(){
82     std::cout<<"Testy dla klasy Point:"<<std::endl;
83     std::cout<<"Test 1: "<<(test1())?"OK":"ERROR"<<" (operator >)"<<std::endl;
84     std::cout<<"Test 2: "<<(test2())?"OK":"ERROR"<<" (operator <)"<<std::endl;
85     std::cout<<"Test 3: "<<(test3())?"OK":"ERROR"<<" (operator <)"<<std::endl;
86     std::cout<<"Test 4: "<<(test4())?"OK":"ERROR"<<" (operator ==)"<<std::endl;
87     std::cout<<"Test 5: "<<(test5())?"OK":"ERROR"<<" (operator !=)"<<std::endl;
88     std::cout<<"Test 6: "<<(test6())?"OK":"ERROR"<<" (operator <=)"<<std::endl;
89     std::cout<<"Test 7: "<<(test7())?"OK":"ERROR"<<" (operator >=)"<<std::endl;
90     std::cout<<"Test 8: "<<(test8())?"OK":"ERROR"<<" (operator =)"<<std::endl;
91
92     return 0;
93 }

```

Plik źródłowy 51: /Struktury/Funkcje/Testy/test_point.cpp

```

1 #!/bin/bash
2
3 NAME="test_point.exe"
4
5 g++ -Wall -o $NAME test_point.cpp ../../Point.cpp

```

```
6 ./ $NAME
7 rm $NAME
```

Plik źródłowy 52: /Struktury/Funkcje/Testy/run_test_point.sh

10.4 test_huffman.cpp

Sprawdzamy poprawność zbioru funkcji *Huffman* poprzez sprawdzenie czy funkcja *Huffman* zwraca poprawne wyniki (przy okazji sprawdzamy wszystkie zdefiniowane funkcje pomocnicze)

```
1 #include "../Huffman.h"
2 #include<iostream>
3 using namespace std;
4
5 bool test1(){
6     vector<char> characters = {'a', 'b', 'c', 'd', 'e', 'f'};
7     vector<int> frequencies = {45, 13, 12, 16, 9, 5};
8
9     vector<HuffmanCode> expectedCodes = {
10         {'a', "0"},
11         {'b', "101"},
12         {'c', "100"},
13         {'d', "111"},
14         {'e', "1101"},
15         {'f', "1100"}
16     };
17
18     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
19
20     return HuffmanEquals(huffmanCodes, expectedCodes);
21 }
22
23 bool test2(){
24     vector<char> characters = {'a', 'b', 'c', 'd', 'e', 'f'};
25     vector<int> frequencies = {1, 1, 1, 1, 1, 1};
26
27     vector<HuffmanCode> expectedCodes = {
28         {'a', "110"},
29         {'b', "00"},
30         {'c', "111"},
31         {'d', "01"},
32         {'e', "101"},
33         {'f', "100"}
34     };
35
36     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
37
38     return HuffmanEquals(huffmanCodes, expectedCodes);
39 }
40
41 bool test3(){
42     vector<char> characters = {'m', 'k', 'x', 'y', 'z'};
43     vector<int> frequencies = {18, 9, 2, 3, 7};
44
45     vector<HuffmanCode> expectedCodes = {
46         {'k', "10"},
47         {'m', "0"},
48         {'x', "1100"},
49         {'y', "1101"},
50         {'z', "111"}
51     };
52
53     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
54
55     return HuffmanEquals(huffmanCodes, expectedCodes);
56 }
```

```

57
58 bool test4(){
59     vector<char> characters = {};
60     vector<int> frequencies = {};
61
62     vector<HuffmanCode> expectedCodes = {};
63
64     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
65
66     return HuffmanEquals(huffmanCodes, expectedCodes);
67 }
68
69 bool test5(){
70     vector<char> characters = {'x', 'y', 'z'};
71     vector<int> frequencies = {};
72
73     vector<HuffmanCode> expectedCodes = {};
74
75     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
76
77     return HuffmanEquals(huffmanCodes, expectedCodes);
78 }
79
80 bool test6(){
81     vector<char> characters = {};
82     vector<int> frequencies = {2, 3, 5, 7};
83
84     vector<HuffmanCode> expectedCodes = {};
85
86     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
87
88     return HuffmanEquals(huffmanCodes, expectedCodes);
89 }
90
91 int main(){
92     std::cout<<"Testy dla funkcji huffman:"<<std::endl;
93     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (normal set)"<<std::endl;
94     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (same freq)"<<std::endl;
95     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (normal set)"<<std::endl;
96     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (both empty)"<<std::endl;
97     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (freq empty)"<<std::endl;
98     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (chars empty)"<<std::endl;
99
100     return 0;
101 }

```

Plik źródłowy 53: /Struktury/Funkcje/Testy/test_huffman.cpp

```

1 #!/bin/bash
2
3 NAME="test_huffman.exe"
4
5 g++ -Wall -o $NAME test_huffman.cpp ../../Huffman.cpp
6 ./ $NAME
7 rm $NAME

```

Plik źródłowy 54: /Struktury/Funkcje/Testy/run_test_huffman.sh

10.5 test_segment_tree.cpp

Testy drzewa przedziałowego sprawdzają zapytania o maksimum na różnych zakresach, obsługę wartości ujemnych, zapytanie jednoelementowe oraz aktualizacje zwiększające i zmniejszające maksimum.

```

1 #include "../SegmentTree.h"
2 #include <iostream>
3

```

```

4 // Test 1: Przykład z wykładu (indeksowanie od 0)
5 bool test1()
6 {
7     SegmentTree t1({5, 2, 7, 1, 3, 6, 4, 8});
8     const int expected = 8;
9     return (t1.query(2, 7) == expected);
10 }
11
12 // Test 2
13 bool test2()
14 {
15     SegmentTree t2({2, 3, -1, 5, -2, 4, 8, 10});
16     const int expected = 5;
17     return (t2.query(2, 4) == expected);
18 }
19
20 // Test 3: Same wartosci ujemne (sprawdzenie czy poprawnie zwraca wartosci ponizej 0)
21 bool test3()
22 {
23     SegmentTree t3({-10, -5, -20, -3, -15});
24     const int expected = -3;
25     return (t3.query(0, 4) == expected);
26 }
27
28 // Test 4: Zapytanie o przedzial jednoelementowy (L == R)
29 bool test4()
30 {
31     SegmentTree t4({1, 5, 9, 2});
32     const int expected = 9;
33     return (t4.query(2, 2) == expected);
34 }
35
36 // Test 5: Test operacji UPDATE (mala wartosc na olbrzymia)
37 bool test5()
38 {
39     SegmentTree t5({1, 2, 3, 4});
40     t5.update(1, 100);
41     const int expected = 100;
42     return (t5.query(0, 3) == expected);
43 }
44
45 // Test 6: Test operacji UPDATE, ktora zmniejsza maksimum
46 bool test6()
47 {
48     SegmentTree t6({5, 20, 10});
49     t6.update(1, 2);
50     const int expected = 10;
51     return (t6.query(0, 2) == expected);
52 }
53
54 int main() {
55     std::cout << "Testy dla drzewa przedzialowego (MaxQuery):" << std::endl;
56     std::cout << "Test 1: " << (test1() ? "OK" : "ERROR") << " (Expected: 8)" << std::endl;
57     std::cout << "Test 2: " << (test2() ? "OK" : "ERROR") << " (Expected: 5)" << std::endl;
58     std::cout << "Test 3: " << (test3() ? "OK" : "ERROR") << " (Expected: -3)" << std::endl;
59     std::cout << "Test 4: " << (test4() ? "OK" : "ERROR") << " (Expected: 9)" << std::endl;
60     std::cout << "Test 5: " << (test5() ? "OK" : "ERROR") << " (Expected: 100 po update)" << std::endl;
61     std::cout << "Test 6: " << (test6() ? "OK" : "ERROR") << " (Expected: 10 po zmniejszeniu max)" << std::endl;
62     return 0;
63 }

```

64 }

Plik źródłowy 55: /Struktury/Funkcje/Testy/test_segment_tree.cpp

10.6 test_class_BorderPatrol.cpp

Testy sprawdzają zachowanie *BorderPatrolSolver* dla pustego zbioru kopalni, jednej kopalni, dwóch kopalni, kwadratu oraz punktu znajdującego się wewnątrz otoczki. Dzięki temu weryfikowana jest zarówno długość patrolu, jak i poprawność wyznaczonej otoczki.

```
1 #include "../BorderPatrolSolver.h"
2 #include "../EPS.h"
3 #include <iostream>
4
5 // Brak kopalni - odległość powinna być 0
6 bool test1() {
7     std::vector<Point> mines = {};
8     BorderPatrolSolver solver(mines);
9
10    double dist = solver.calculatePatrolDistance();
11    return cmpDouble(dist, 0.0) == 0.0;
12 }
13
14 // Jedna kopalnia - nie ma trasy do okrążenia
15 bool test2() {
16     std::vector<Point> mines = {Point(1, 1)};
17     BorderPatrolSolver solver(mines);
18
19     double dist = solver.calculatePatrolDistance();
20     return cmpDouble(dist, 0.0) == 0.0;
21 }
22
23 // Dwie kopalnie - trasa to odcinek tam i z powrotem
24 bool test3() {
25     std::vector<Point> mines = {Point(0, 0), Point(3, 0)};
26     BorderPatrolSolver solver(mines);
27
28     double dist = solver.calculatePatrolDistance();
29     return cmpDouble(dist, 6.0) == 0;
30 }
31
32 // Cztery kopalnie tworzą kwadrat 4x4 - obwód = 16
33 bool test4() {
34     std::vector<Point> mines = {
35         Point(0, 0), Point(4, 0), Point(4, 4), Point(0, 4)
36     };
37     BorderPatrolSolver solver(mines);
38
39     double dist = solver.calculatePatrolDistance();
40     return cmpDouble(dist, 16.0) == 0;
41 }
42
43 // Jeden punkt wewnątrz kwadratu - nie wpływa na otoczkę ani obwód
44 bool test5() {
45     std::vector<Point> mines = {
46         Point(0, 0), Point(4, 0), Point(4, 4), Point(0, 4), Point(2, 2)
47     };
48     BorderPatrolSolver solver(mines);
49
50     double dist = solver.calculatePatrolDistance();
51     return cmpDouble(dist, 16.0) == 0;
52 }
53
54 // Sprawdzamy czy calculateConvexHull zwraca właściwe punkty
55 bool test6() {
56     std::vector<Point> mines = {
57         Point(0, 0), Point(4, 0), Point(4, 4), Point(0, 4), Point(2, 2)
58     };
59     BorderPatrolSolver solver(mines);
60
61     double dist = solver.calculatePatrolDistance();
62     return cmpDouble(dist, 16.0) == 0;
63 }
```



```

58     };
59     BorderPatrolSolver solver(mines);
60     std::vector<Point> hull = solver.calculateConvexHull();
61
62     // (2,2) nie powinien być w otoczce
63     for(const Point& p : hull)
64         if(p == Point(2, 2)) return false;
65     return hull.size() == 4;
66 }
67
68 int main() {
69     std::cout << "Testy dla BorderPatrolSolver:" << std::endl;
70
71     std::cout << "Test 1: " << (test1() ? "OK" : "ERROR") << " (pusty zbior)" << std::endl;
72     std::cout << "Test 2: " << (test2() ? "OK" : "ERROR") << " (jedna kopalnia)" << std::endl;
73     std::cout << "Test 3: " << (test3() ? "OK" : "ERROR") << " (dwie kopalnie)" << std::endl;
74     std::cout << "Test 4: " << (test4() ? "OK" : "ERROR") << " (kwadrat)" << std::endl;
75     std::cout << "Test 5: " << (test5() ? "OK" : "ERROR") << " (punkt wewnątrz)" << std::endl;
76     std::cout << "Test 6: " << (test6() ? "OK" : "ERROR") << " (poprawność otoczki)" << std::endl;
77
78     return 0;
79 }

```

Plik źródłowy 56: /Struktury/Funkcje/Testy/test_class_BorderPatrol.cpp

10.7 test_work_assignment_solver.cpp

Testy *WorkAssignmentSolver* sprawdzają prosty przydział jeden do jednego, priorytet preferowanego surowca względem krótszej drogi, obsługę pojemności kopalni oraz sytuację, w której krasnal nie ma możliwego przydziału.

```

1  #include "../WorkAssignmentSolver.h"
2
3  #include <cmath>
4  #include <iostream>
5  #include <vector>
6
7  bool nearDouble(double a, double b) {
8      return std::fabs(a - b) < 0.000001;
9  }
10
11 bool testSimpleAssignment() {
12     std::vector<Krasnal> dwarves = {
13         Krasnal(1, Point(0, 0), {"zloto"}, "zloto", false)
14     };
15     std::vector<Kopalnia> mines = {
16         Kopalnia(10, Point(3, 4), "zloto", 1, {})
17     };
18
19     WorkAssignmentSolver solver(dwarves, mines);
20     WorkAssignmentResult result = solver.solve();
21
22     return result.assignedCount == 1 &&
23         result.preferredAssignedCount == 1 &&
24         result.unassignedDwarfIds.empty() &&
25         result.assignments.size() == 1 &&
26         result.assignments[0].dwarfId == 1 &&
27         result.assignments[0].mineId == 10 &&
28         nearDouble(result.totalDistance, 5.0) &&
29         solver.getMines()[0].getAssignedDwarves().size() == 1;
30 }

```

```

31
32 bool testPreferenceBeforeDistance() {
33     std::vector<Krasnal> dwarves = {
34         Krasnal(1, Point(0, 0), {"zloto", "miedz"}, "zloto", false)
35     };
36     std::vector<Kopalnia> mines = {
37         Kopalnia(10, Point(10, 0), "zloto", 1, {}),
38         Kopalnia(11, Point(1, 0), "miedz", 1, {})
39     };
40
41     WorkAssignmentSolver solver(dwarves, mines);
42     WorkAssignmentResult result = solver.solve();
43
44     return result.assignedCount == 1 &&
45           result.preferredAssignedCount == 1 &&
46           result.assignments.size() == 1 &&
47           result.assignments[0].mineId == 10;
48 }
49
50 bool testMineCapacity() {
51     std::vector<Krasnal> dwarves = {
52         Krasnal(1, Point(0, 0), {"wegiel"}, "wegiel", false),
53         Krasnal(2, Point(1, 0), {"wegiel"}, "wegiel", false)
54     };
55     std::vector<Kopalnia> mines = {
56         Kopalnia(10, Point(0, 1), "wegiel", 2, {})
57     };
58
59     WorkAssignmentSolver solver(dwarves, mines);
60     WorkAssignmentResult result = solver.solve();
61
62     return result.assignedCount == 2 &&
63           result.preferredAssignedCount == 2 &&
64           result.unassignedDwarfIds.empty() &&
65           solver.getMines()[0].getAssignedDwarves().size() == 2;
66 }
67
68 bool testUnassignedDwarf() {
69     std::vector<Krasnal> dwarves = {
70         Krasnal(1, Point(0, 0), {"wegiel"}, "wegiel", false)
71     };
72     std::vector<Kopalnia> mines = {
73         Kopalnia(10, Point(0, 1), "zloto", 1, {})
74     };
75
76     WorkAssignmentSolver solver(dwarves, mines);
77     WorkAssignmentResult result = solver.solve();
78
79     return result.assignedCount == 0 &&
80           result.preferredAssignedCount == 0 &&
81           result.unassignedDwarfIds.size() == 1 &&
82           result.unassignedDwarfIds[0] == 1 &&
83           result.assignments.empty();
84 }
85
86 int main() {
87     const bool test1 = testSimpleAssignment();
88     const bool test2 = testPreferenceBeforeDistance();
89     const bool test3 = testMineCapacity();
90     const bool test4 = testUnassignedDwarf();
91
92     std::cout << "Testy dla WorkAssignmentSolver:" << std::endl;
93     std::cout << "Test 1: " << (test1 ? "OK" : "ERROR") << " (prosty przydzial)" <<
94     std::endl;
95     std::cout << "Test 2: " << (test2 ? "OK" : "ERROR") << " (preferencja przed
96     dystansem)" << std::endl;
97     std::cout << "Test 3: " << (test3 ? "OK" : "ERROR") << " (pojemnosc kopalni)" <<
98     std::endl;

```

```

96     std::cout << "Test 4: " << (test4 ? "OK" : "ERROR") << " (brak mozliwego
przydzialu)" << std::endl;
97
98     return (test1 && test2 && test3 && test4) ? 0 : 1;
99 }

```

Plik źródłowy 57: /Struktury/WorkAssignmentSolver/Testy/test_work_assignment_solver.cpp

10.8 test_guard_command_solver.cpp

Testy *GuardCommandSolver* sprawdzają wybór najgłośniejszego strażnika na całym zakresie, na podzakresie, rozstrzyganie remisu przez wybór pierwszego strażnika, aktualizację głośności, błędny zakres oraz pustą listę strażników.

```

1 #include "../GuardCommandSolver.h"
2
3 #include <iostream>
4 #include <vector>
5
6 std::vector<Straznik> createGuards() {
7     return {
8         Straznik(100, 14),
9         Straznik(101, 9),
10        Straznik(102, 18),
11        Straznik(103, 13),
12        Straznik(104, 18),
13        Straznik(105, 11)
14    };
15 }
16
17 bool testWholeRange() {
18     GuardCommandSolver solver(createGuards());
19     GuardCommandResult result = solver.findLoudestGuard(0, solver.size() - 1);
20
21     return result.found &&
22            result.index == 2 &&
23            result.guardId == 102 &&
24            result.loudness == 18;
25 }
26
27 bool testSubRange() {
28     GuardCommandSolver solver(createGuards());
29     GuardCommandResult result = solver.findLoudestGuard(3, 5);
30
31     return result.found &&
32            result.index == 4 &&
33            result.guardId == 104 &&
34            result.loudness == 18;
35 }
36
37 bool testTieReturnsFirstGuard() {
38     GuardCommandSolver solver({
39         Straznik(1, 7),
40         Straznik(2, 7),
41         Straznik(3, 5)
42     });
43
44     GuardCommandResult result = solver.findLoudestGuard(0, 2);
45
46     return result.found &&
47            result.index == 0 &&
48            result.guardId == 1 &&
49            result.loudness == 7;
50 }
51
52 bool testUpdateLoudness() {
53     GuardCommandSolver solver(createGuards());

```

```

54
55     if (!solver.updateLoudness(1, 20)) {
56         return false;
57     }
58
59     GuardCommandResult result = solver.findLoudestGuard(0, solver.size() - 1);
60
61     return result.found &&
62           result.index == 1 &&
63           result.guardId == 101 &&
64           result.loudness == 20;
65 }
66
67 bool testInvalidRange() {
68     GuardCommandSolver solver(createGuards());
69     GuardCommandResult result = solver.findLoudestGuard(4, 2);
70
71     return !result.found;
72 }
73
74 bool testEmptyGuards() {
75     GuardCommandSolver solver({});
76     GuardCommandResult result = solver.findLoudestGuard(0, 0);
77
78     return solver.size() == 0 && !result.found;
79 }
80
81 int main() {
82     const bool test1 = testWholeRange();
83     const bool test2 = testSubRange();
84     const bool test3 = testTieReturnsFirstGuard();
85     const bool test4 = testUpdateLoudness();
86     const bool test5 = testInvalidRange();
87     const bool test6 = testEmptyGuards();
88
89     std::cout << "Testy dla GuardCommandSolver:" << std::endl;
90     std::cout << "Test 1: " << (test1 ? "OK" : "ERROR") << " (caly zakres)" << std::endl;
91     std::cout << "Test 2: " << (test2 ? "OK" : "ERROR") << " (podzakres)" << std::endl;
92     std::cout << "Test 3: " << (test3 ? "OK" : "ERROR") << " (remis wybiera pierwszego)" << std::endl;
93     std::cout << "Test 4: " << (test4 ? "OK" : "ERROR") << " (aktualizacja glosnosci)" << std::endl;
94     std::cout << "Test 5: " << (test5 ? "OK" : "ERROR") << " (bledny zakres)" << std::endl;
95     std::cout << "Test 6: " << (test6 ? "OK" : "ERROR") << " (brak straznikow)" << std::endl;
96
97     return (test1 && test2 && test3 && test4 && test5 && test6) ? 0 : 1;
98 }

```

Plik źródłowy 58: /Struktury/GuardCommandSolver/Testy/test_guard_command_solver.cpp

10.9 Uruchamianie testów

Główne testy projektu są zintegrowane z CMake i CTest. Po zbudowaniu projektu można uruchomić je poleceniem:

```
1 ctest --test-dir cmake-build-debug --output-on-failure
```

Aktualnie w CTest znajdują się testy parsera danych, minimalnego kosztu maksymalnego przepływu, patrolu granicy, przydziału pracy oraz wyboru strażnika.

11 Prezentacja działania

11.1 Główny przepływ programu

Główny program łączy przygotowane moduły w jeden przepływ danych. Za koordynację odpowiada *KingdomManager*, który kolejno:

1. *DataLoader* wczytuje krasnale, kopalnie i strażników z pliku wejściowego.
2. *WorkAssignmentSolver* przydziela krasnale do kopalni za pomocą minimalnego kosztu maksymalnego przepływu.
3. Z wyniku przydziału wybierane są tylko aktywne kopalnie, czyli te, do których trafił przynajmniej jeden krasnal.
4. *BorderPatrolSolver* wyznacza otoczkę wypukłą aktywnych kopalni i długość trasy patrolu.
5. *GuardCommandSolver* odpowiada na zapytania o najgłośniejszego strażnika na wskazanym odcinku granicy.

11.2 Przykładowy wynik dla Dane/dane.txt

Dla aktualnego pliku wejściowego program wczytuje większy zestaw danych: 30 krasnali, 12 kopalni oraz 20 strażników. Następnie wypisuje przydział pracy, listę ewentualnych krasnali bez przydziału, aktywne kopalnie, punkty trasy patrolu oraz przykładowe zapytania dla obrony granicy.

```
1 Krasnale: 30
2 Kopalnie: 12
3 Straznicy: 20
4
5 Przydzial pracy:
6 Krasnal ... -> kopalnia ... (...), dystans: ... [preferowany surowiec]
7 ...
8
9 Przydzielono: .../30
10 Przydzialy preferowane: ...
11 Laczny dystans: ...
12
13 Aktywne kopalnie:
14 Kopalnia ... (...), krasnale: ...
15 ...
16
17 Punkty trasy patrolu:
18 (...)
19 ...
20 Dlugosc trasy patrolu: ...
21
22 Obrona granicy:
23 Najglosniejszy na calej trasie: straznik ... na pozycji ..., glosnosc: ...
24 Odcinek ataku [1, 3]: straznik ... na pozycji ..., glosnosc: ...
```

Plik źródłowy 59: Przykładowe wyjście programu